SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

Django

DRF 1

마이스터고

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

REST API

- API
- REST API
- 자원의 식별
- 자원의 행위
- 자원의 표현
- JSON 데이터 응답

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

DRF with Single Model

- Django REST Framework
- Serializer

CRUD with ModelSerializer

- GET method - 조회
- POST method - 생성
- DELETE method - 삭제
- PUT method - 수정

참고

- raise_exception

학습 시작

배달 앱에서 음식을 주문할 때, 우리는 가게에 직접 전화하지 않아도 됩니다!

1. 배달 앱이 [redacted]를 통해 가게에 주문 정보를 전달하고
2. 가게는 그 정보를 받아서 [redacted]형식으로 “주문 확인”을 응답합니다.
3. 우리는 어떤 메뉴를 주문했는지 [redacted]요청으로 다시 확인할 수도 있어요

- 오늘은 이렇게 다양한 서비스들이 서로 약속된 방법으로 대화하는 방식, REST API를 배워볼 거예요
- 복잡한 설명 없이, 주소를 정하고(URI), 행동(GET, POST, DELETE)을 선택하고, 결과를 받아보는 경험을 할 수 있어요
- 내가 만든 서버가 스마트폰 앱처럼 응답하도록 바꿔보는 실습도 함께 진행해볼 거예요

Objectives

학습 목표

- ✓ REST API의 개념과 구성 요소(URI, Method, 표현 방식)를 이해할 수 있다.
- ✓ HTTP 요청 방식(GET, POST, PUT, DELETE)의 역할을 설명할 수 있다.
- ✓ Django REST Framework를 이용해 간단한 API 서버를 구축할 수 있다.
- ✓ ModelSerializer를 사용하여 데이터를 JSON으로 변환하고 응답할 수 있다.
- ✓ API 요청에 따른 응답 코드를 이해하고 적절하게 처리할 수 있다.
- ✓ Postman을 이용해 API 서버를 테스트하고 응답 결과를 확인할 수 있다.
- ✓ partial 인자 사용 등 실제 API 수정 요청에서 발생할 수 있는 상황을 처리할 수 있다

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

REST API

API

API 정의

API

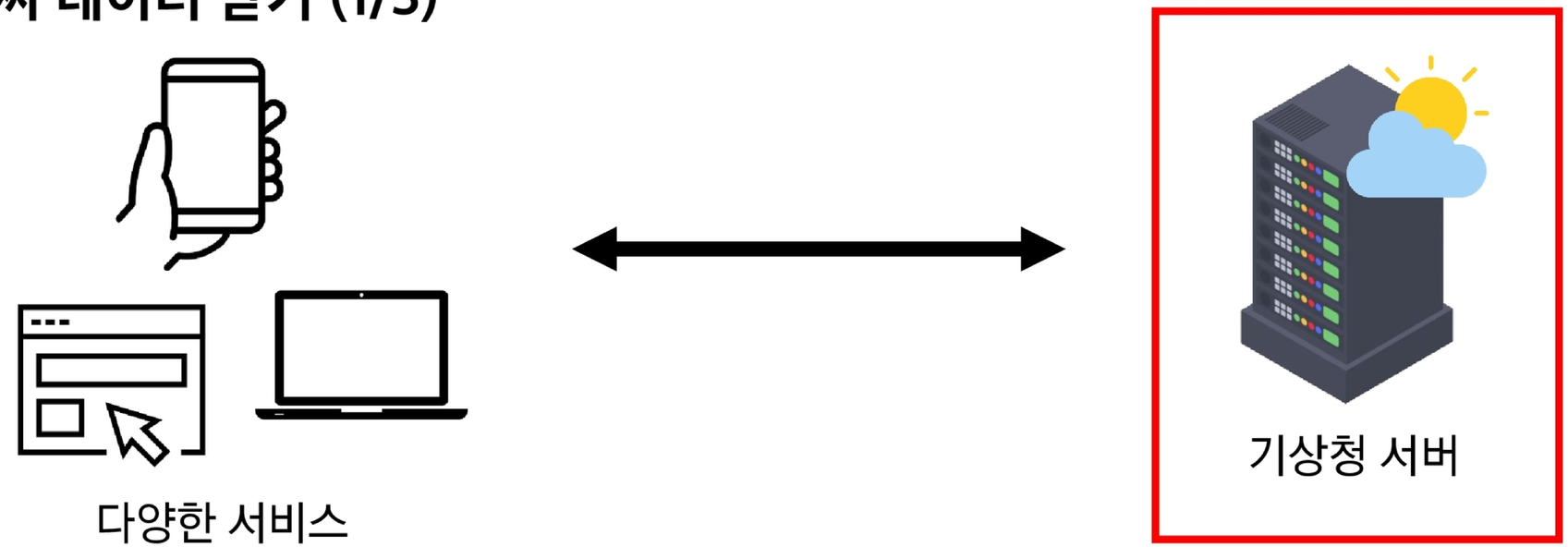
Application Programming Interface

두 소프트웨어가 서로 **통신**
할 수 있게 하는 **메커니즘**

➢ 클라이언트-서버처럼 서로 다른 프로그램에서 요청과 응답을 받을 수 있도록 만든 체계

하나의 프로그램이 다른 프로그램에게 정보를 보내거나 요청할 때,
서로 이해할 수 있는 공통된 규칙과 형식이 필요합니다. 이 역할을 하는 것이 바로 API입니다.

API 예시 - 날씨 데이터 받기 (1/3)

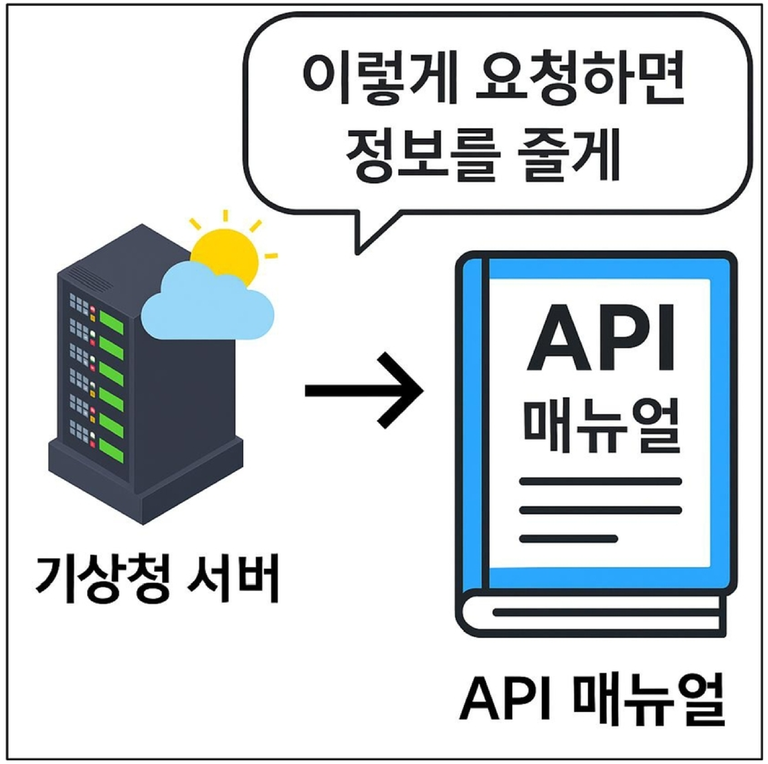


<그림1_Django_DRF 1_API 예시 1>

- 기상 데이터가 들어있는 기상청의 서버
- 스마트폰의 날씨 앱, 웹 사이트의 날씨 정보 등 **다양한 서비스들이** 이 기상청 서버에 데이터를 요청보내고 응답 받음

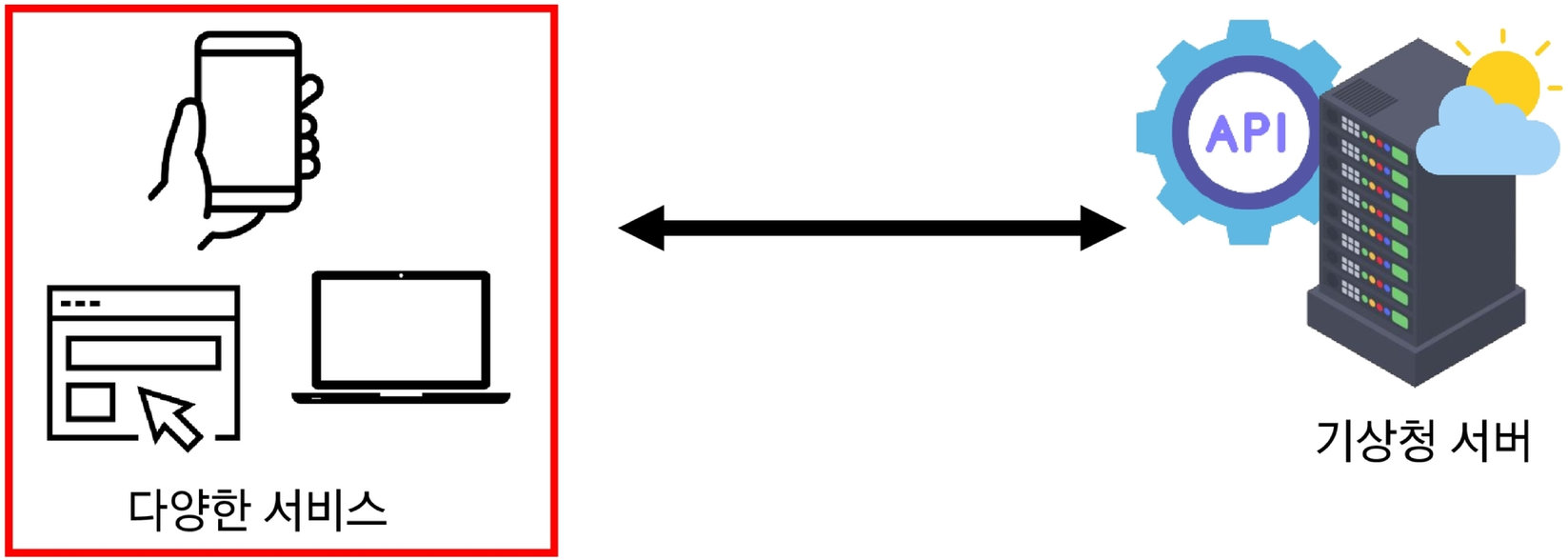
API 예시 - 날씨 데이터 받기 (2/3)

- 날씨 데이터를 얻으려면?
  - 기상청 서버에는 날씨 정보를 얻고 싶으면 이런 식으로 요청해야 한다는 지정된 형식들이 작성되어 있음
  - 지역, 날짜, 조회할 내용들(온도, 바람 등)을 제공하는 매뉴얼



<그림2_Django_DRF 1_API 예시 메뉴얼>

API 예시 - 날씨 데이터 받기 (3/3)



<그림3_Django_DRF 1_API 예시 2>

- 소프트웨어와 소프트웨어 간 지정된 정의(형식)으로 소통하는 수단 → API
  - “이렇게 요청을 보내면, 이렇게 정보를 제공 해줄 것이다”라는 매뉴얼

➢스마트폰의 날씨 앱은 기상청에서 제공하는 API를 통해

기상청 시스템과 대화하여 매일 최신 날씨 정보를 표시 할 수 있음

API 역할

- 예를 들어 우리 집 냉장고에 전기를 공급해야 한다고 가정해보자
- 우리는 그냥 냉장고의 플러그를 소켓에 꽂으면 제품이 작동한다.
- 중요한 것은 우리가 가전 제품에
  “전기를 공급하기 위해 직접 배선을 하지 않는다”는 것이다.
- 이는 매우 위험하면서도 비효율적인 일이기 때문이다.

➢ 복잡한 코드를 추상화하여 대신 사용할 수 있는 몇 가지 더 쉬운 구문을 제공

API
두 소프트웨어가 서로 통신
할 수 있게 하는 메커니즘

<개념1_Django_DRF 1_API 정의>

Web API

- 웹 서버 또는 웹 브라우저를 위한 API
- 현대 웹 개발은 하나부터 열까지 직접 개발하기보다 여러 Open API 들을 활용
- 대표적인 Third Party Open API 서비스 목록
  - Youtube API
  - Google Map API
  - Naver Papago API
  - Kakao Map API

Open API
누구나 접근할 수 있도록 공개된, 외부 소프트웨어와 통신하기 위한 인터페이스

<개념2_Django_DRF 1_Open API 정의>

Third Party
직접 개발하지 않은 외부의 서비스나 소프트웨어를 제공하거나 활용하는 주체

<개념3_Django_DRF 1_Third Party 정의>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

REST API

REST 란?

REST

Representational State Transfer

API Server를 개발하기 위한 일종의
소프트웨어 설계 방법론

※ 엄격한 규칙을 의미하는 것은 아님!

건축 설계를 생각해봅시다.

누구는 창문부터 만들고, 누구는 지붕부터 올리면 공사가 복잡해지겠죠?

그래서 집을 지을 때는 기초-골조-내장-마감처럼 정해진 순서를 따릅니다.

REST도 마찬가지입니다.

API마다 제각각인 구조를 정리하고, 누구나 예측 가능한 방식으로 통신할 수 있도록 설계 기준을 제안한 것이 바로 REST입니다.

RESTful API 란?

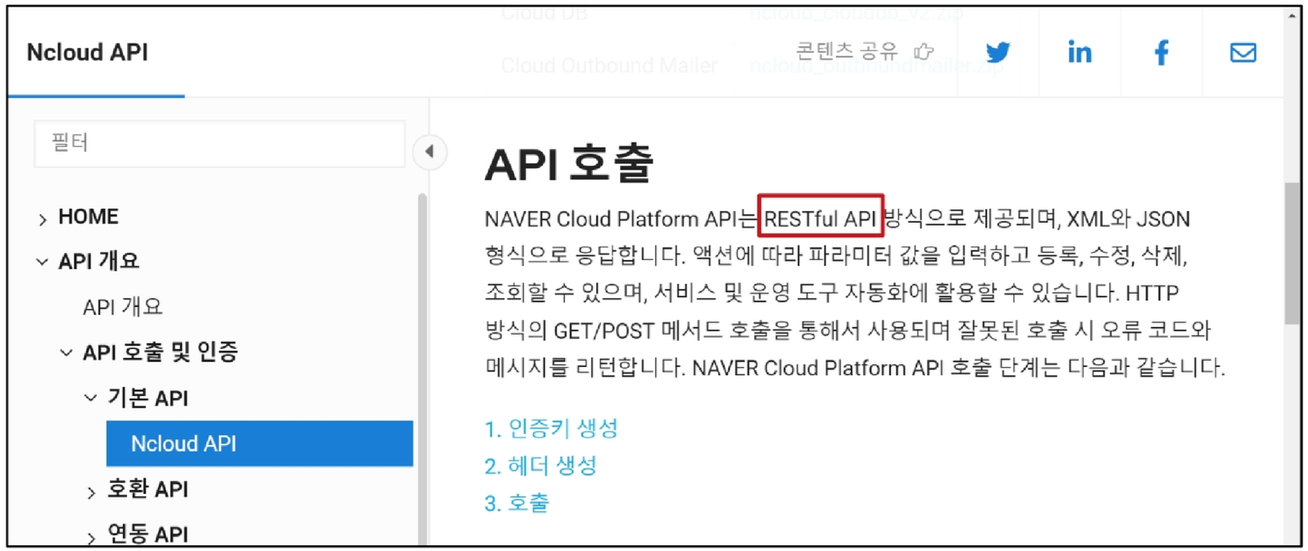
RESTful API

"자원을 정의"하고
"자원에 대한 주소를 지정"하는
전반적인 방법을 서술

REST 원리를 따르는 시스템을 RESTful 하다고 부릅니다.

"각각 API 서버 구조를 작성하는 모습이 너무 다르니 어느정도 약속을 만들어서 다같이 통일해서 쓰자!"

REST API 실제 활용 예시



Naver Cloud API

<그림5_Django_DRF 1_Naver Cloud API>

Kakao Login API

<그림4_Django_DRF 1_Kakao Login API>

※ REST API는 시스템 간 표준화된 통신 구조를 제공함으로써, 서로 다른 기술 스택 간에도 연동이 가능

REST에서 자원을 정의하고 주소를 지정하는 방법

- 자원의 “식별”
  - URI
- 자원의 “행위”
  - HTTP Methods
- 자원의 “표현”
  - JSON 데이터
    (궁극적으로 표현되는 데이터 결과물)

API
두 소프트웨어가 서로 통신할 수 있게 하는 메커니즘

<개념1_Django_DRF 1_API 정의>

REST API
REST라는 설계 디자인 약속을 지켜 구현한 API

<개념4_Django_DRF 1_REST API 정의>

TIP

- **URL만 보고도 무슨 데이터를, 어떤 방식으로 처리할지 예측할 수 있어야 합니다.**
- **개발 시에는 항상 "자원 중심 + 동작 명확화 + 일관된 응답 포맷"을 기준으로 설계하세요.**

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

자원의 식별

URI 정의

URI

Uniform Resource Identifier (통합 자원 식별자)

인터넷에서 리소스(자원)를 식별하는 문자열

가장 일반적인 URI는 웹 주소로 알려진 **URL**

자원 (Resource)
웹에서 고유한 주소를 통해 접근할 수 있는 데이터나 기능의 대상

<개념5_Django_자원 정의>

URL 정의

URL

Uniform Resource Locator (통합 자원 위치)

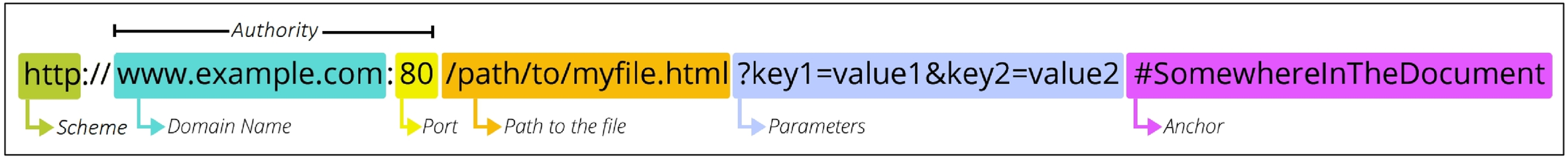
웹에서 주어진 리소스의 주소

네트워크 상에 리소스가 어디 있는지를 알려주기 위한 약속

URL 정의

URL

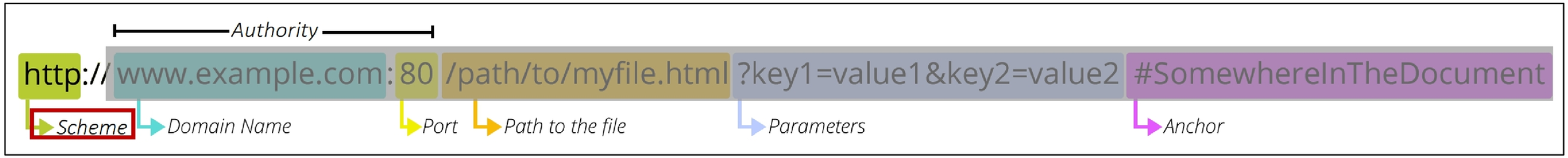
Uniform Resource Locator (통합 자원 위치)



<그림6_Django_DRF 1_URL 예시>

Scheme (or Protocol)

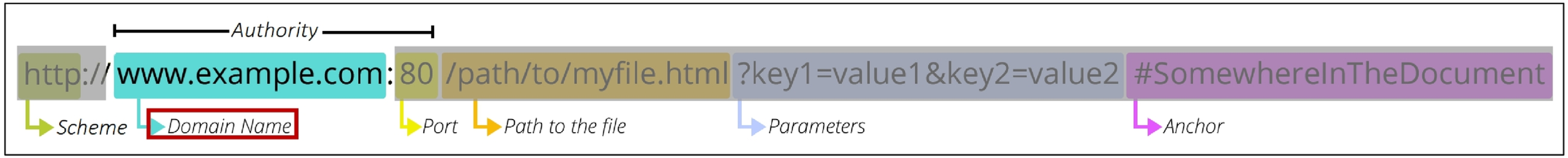
- 브라우저가 리소스를 요청하는 데 사용해야 하는 규약
- URL의 첫 부분은 브라우저가 어떤 규약을 사용하는지를 나타냄
- 기본적으로 웹은 http(s)를 요구
  - 메일을 열기위한 mailto:, 파일을 전송하기 위한 ftp: 등 다른 프로토콜도 존재



<그림7_Django_DRF 1_URL 예시 Scheme>

Domain Name

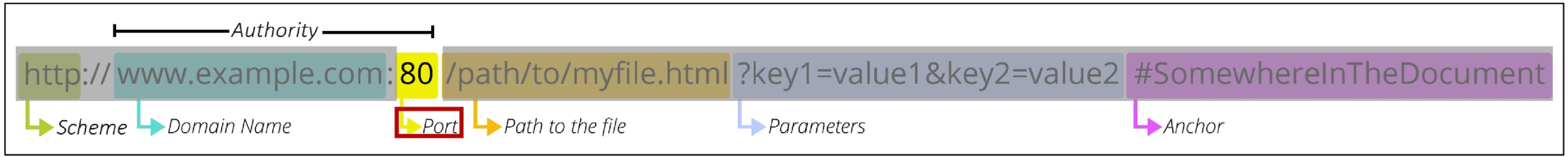
- 요청 중인 웹 서버를 나타냄
- 어떤 웹 서버가 요구되는 지를 가리키며 직접 IP 주소를 사용하는 것도 가능하지만, 사람이 외우기 어렵기 때문에 주로 Domain Name으로 사용
- 예) 도메인 google.com의 IP 주소는 142.251.42.142



<그림8_Django_DRF 1_URL 예시 Domain Name>

Port

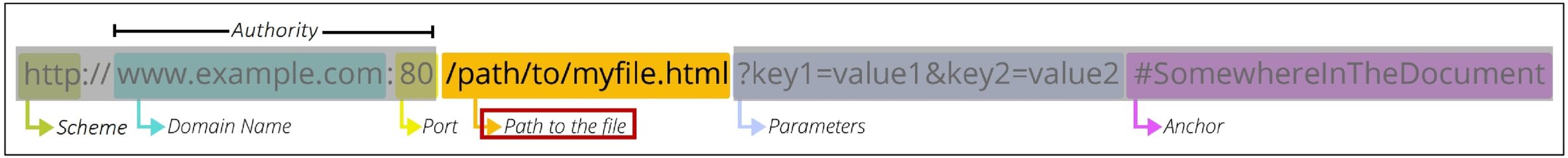
- 웹 서버의 리소스에 접근하는데 사용되는 기술적인 문(Gate)
- HTTP 프로토콜의 표준 포트
  - HTTP - 80
  - HTTPS - 443
- 표준 포트만 작성 시 생략 가능



<그림9_Django_DRF 1_URL 예시 Port>

Path

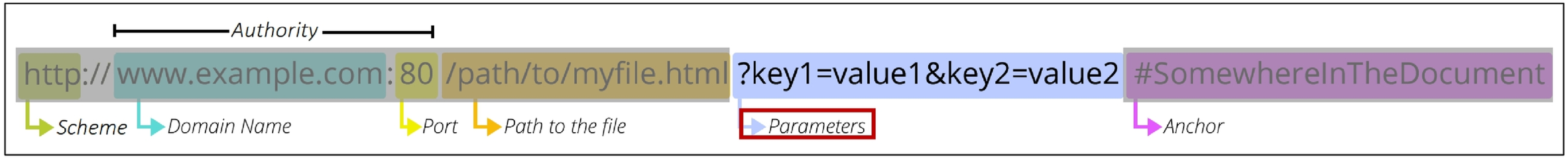
- 웹 서버의 리소스 경로
- 초기에는 실제 파일이 위치한 물리적 위치를 나타냈지만,
  오늘날은 실제 위치가 아닌 추상화된 형태의 구조를 표현
- 예) /articles/create/라는 주소가 실제 articles 폴더안에 create 폴더안을 나타내는 것은 아님



<그림10_Django_DRF 1_URL 예시 Path>

Parameters

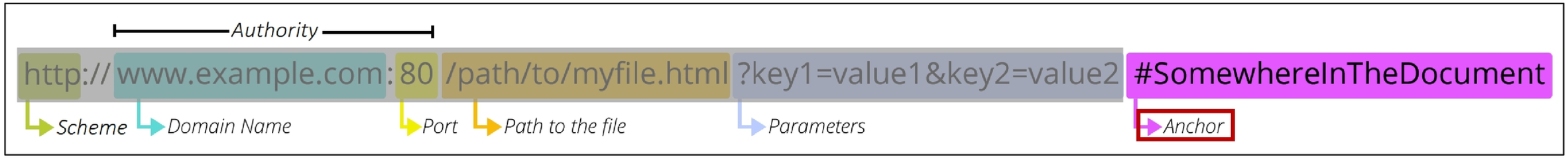
- 웹 서버에 제공하는 추가적인 데이터
- '&' 기호로 구분되는 key-value 쌍 목록
- 서버는 리소스를 응답하기 전에 이러한 파라미터를 사용하여 추가 작업을 수행할 수 있음



<그림11_Django_DRF 1_URL 예시 Parameters>

Anchor

- 일종의 “북마크”를 나타내며 브라우저에 해당 지점에 있는 콘텐츠를 표시
- ‘#’ (fragment identifier, 부분 식별자) 이후 부분은 서버에 전송되지 않음
- **https://docs.djangoproject.com/en/5.2/intro/install/#quick-install-guide** 요청에서 #quick-install-guide는 서버에 전달되지 않고 브라우저에게 해당 지점으로 이동할 수 있도록 함



<그림12_Django_DRF 1_URL 예시 Anchor>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

자원의 행위

methods 정의

HTTP Request Methods

리소스에 대한 행위
즉, **수행하고자 하는 동작**을 정의

"이 주소로 물건 좀 보내주세요" → {{ POST }}
"방금 보낸 물건 도착했나요?" → {{ GET }}
"그 물건 취소할게요" → {{ DELETE }}
"받는 사람 정보를 통째로 새로 적을게요" → {{ PUT }}

HTTP Request Methods 종류

- GET
  - 서버에 리소스의 표현을 요청
  - GET을 사용하는 요청은 데이터만 검색해야 함
- POST
  - 데이터를 지정된 리소스에 제출
  - 서버의 상태를 변경
- PUT
  - 요청한 주소의 리소스를 수정
- DELETE
  - 지정된 리소스를 삭제

HTTP Request Methods
리소스에 대한 행위
즉, 수행하고자 하는 동작을 정의

<개념6_Django_DRF 1_HTTP Request Methods>

status code

HTTP response status codes

특정 HTTP 요청이 **성공적으로 완료 되었는지** 여부를 나타냄

클라이언트가 서버에 요청을 보내면, 서버는 요청이 성공했는지 실패했는지를 숫자로 알려줍니다.
클라이언트는 이 코드를 보고 어떤 일이 일어났는지 판단할 수 있습니다.

HTTP response status codes

- Informational responses (100-199)
  - 요청을 계속 진행 중이라는 중간 응답
- Successful responses (200-299)
  - 요청이 정상적으로 처리되었음을 의미
- Redirection messages (300-399)
  - 요청한 리소스가 다른 위치로 옮겨졌을 때 사용
- Client error responses (400-499)
  - 클라이언트 요청에 문제가 있을 때 반환
- Server error responses (500-599)
  - 서버 내부의 문제로 요청을 처리하지 못했을 때 사용

status codes
요청 처리 결과를 알려주는 숫자 응답 신호

<개념7_Django_DRF 1_status codes 정의>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

자원의 표현

그동안 서버가 응답(자원을 표현)했던 것

- 지금까지 Django 서버는 사용자에게 페이지(html)만 응답하고 있었음
- 하지만 서버가 응답할 수 있는 것은 페이지 뿐만 아니라 **다양한 데이터 타입을 응답할 수 있음**
- REST API는 이 중에서도 JSON 타입으로 응답하는 것을 권장

※ JSON은 데이터만을 전달하기 위한 최소한의 형식

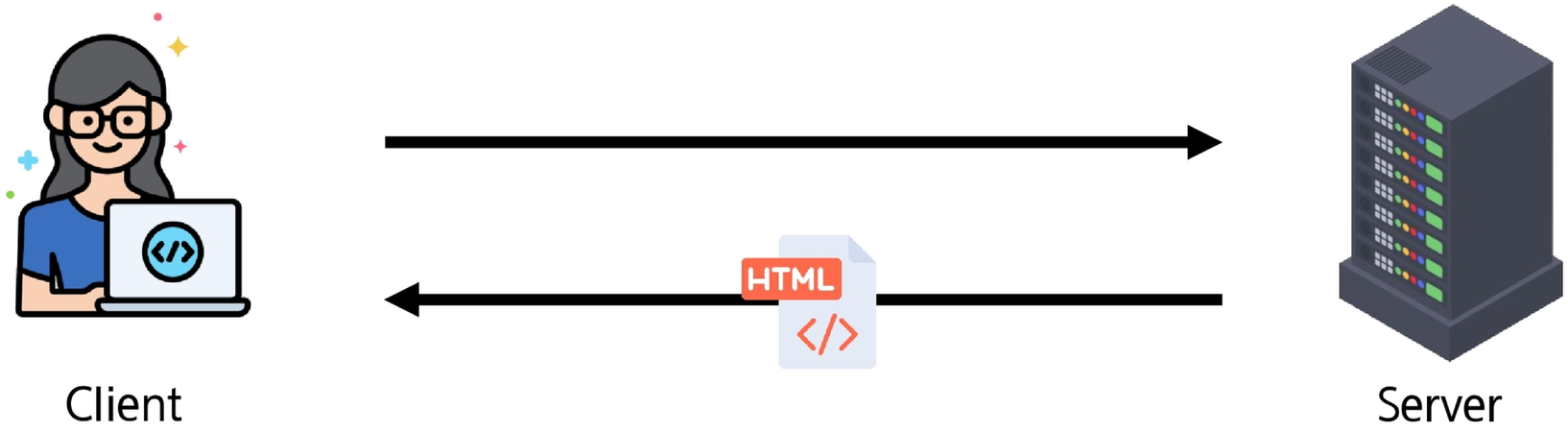
※ 어떤 클라이언트와도 언어와 플랫폼에 독립적으로 통신할 수 있게 해줌

JSON
데이터를 구조화된 텍스트 형태로 표현하는 형식

<개념8_Django_DRF 1_JSON>

응답 데이터 타입의 변화 (1/4)

- 페이지(html)만을 응답하는 서버

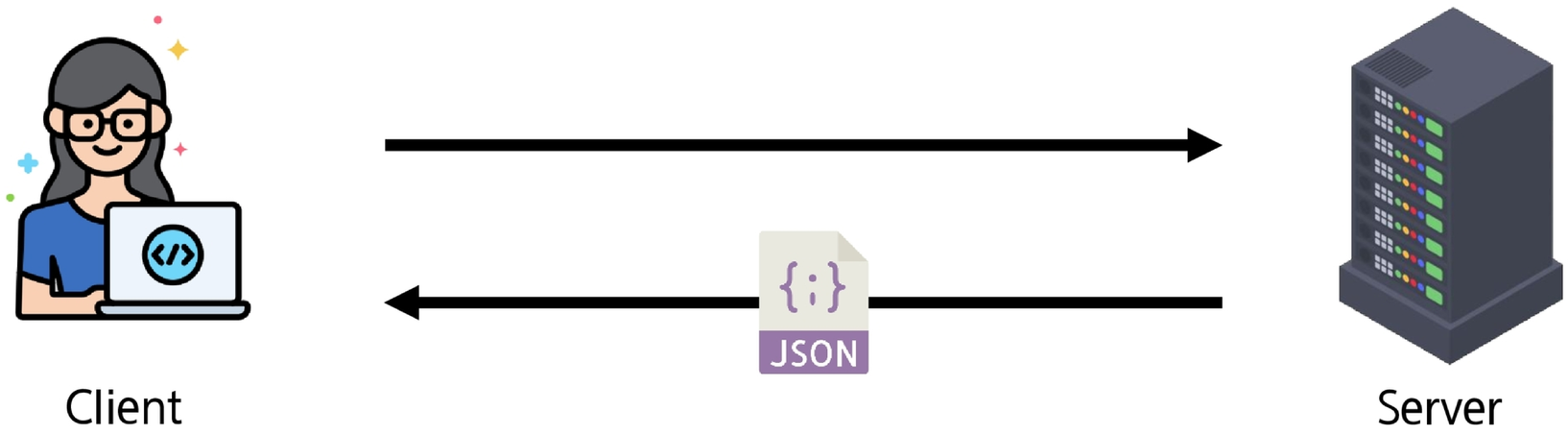


<그림13_Django_DRF 1_HTML만을 응답하는 서버>

사용자가 웹 브라우저를 통해 요청하면, 서버는 템플릿을 렌더링한 HTML을 반환

응답 데이터 타입의 변화 (2/4)

- 이제는 JSON 데이터를 응답하는 REST API 서버로의 변환

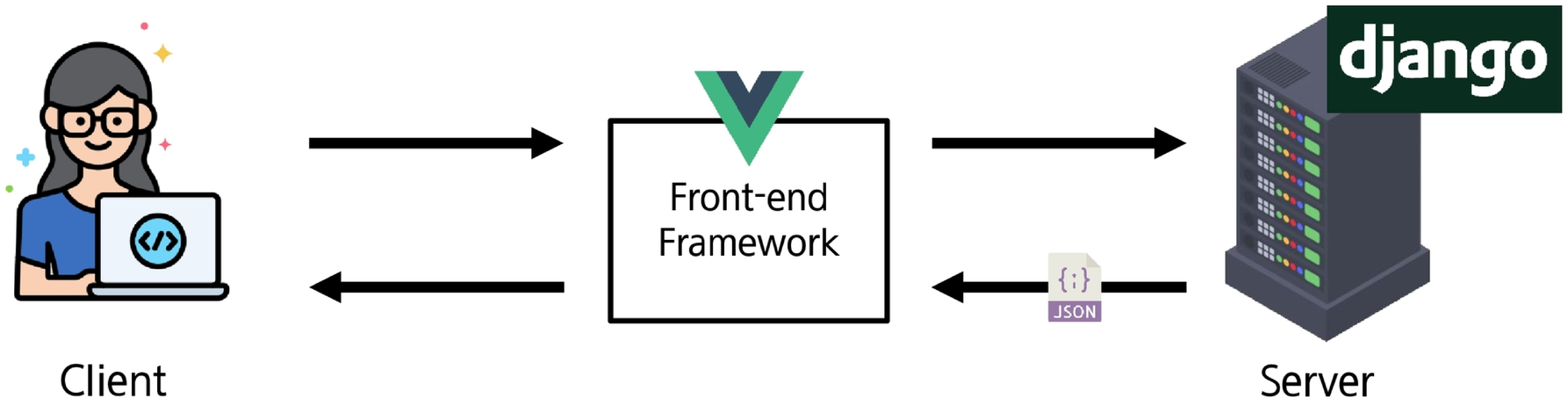


<그림14_Django_DRF 1_JSON을 응답하는 REST API 서버>

- ※ 서버는 HTML 페이지를 만들지 않고, JSON 데이터만 응답하는 방식으로 동작할 수 있음
- ※ HTML 대신 JSON만 전달하므로, 응답 용량이 줄고, 화면을 그리는 일은 클라이언트가 맡게 됨

응답 데이터 타입의 변화 (3/4)

- Django는 더 이상 화면(Template)을 그리는 역할을 하지 않음
- 대신, 오직 데이터(JSON)만을 응답하는 API 서버로 역할이 변화

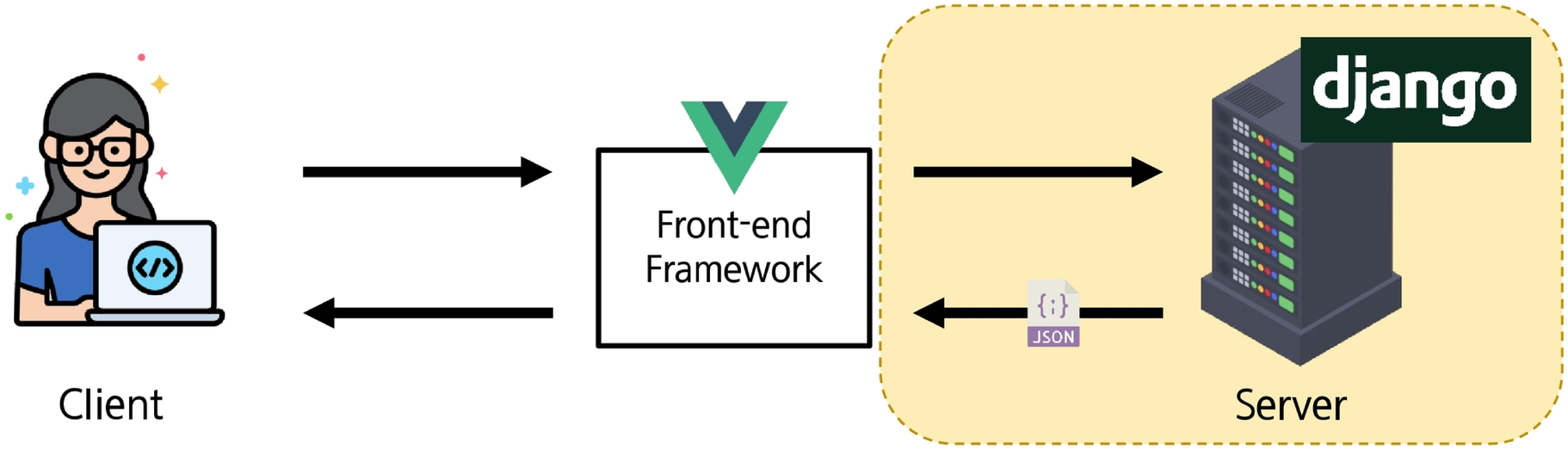


<그림15_Django_DRF 1_FrontEnd와 BackEnd 분리 예시>

※ 이제 화면은 Vue 같은 Front-end 프레임워크가 전담하며,
전통적인 MTV 구조(서버가 화면까지 그리는 방식)에서 'Front-end/Back-end 분리' 구조로의 전환을 의미함

응답 데이터 타입의 변화 (4/4)

- 이제부터 Django를 사용해 RESTful API 서버를 구축할 것



<그림16_Django_DRF 1_Django를 활용한 REST API 구축>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

JSON 데이터 응답

사전 준비 (1/4)

- 사전 제공된 99-json-response-practice 기반 시작
- 가상 환경 생성, 활성화 및 패키지 설치

```
$ python -m venv venv
$ source venv/Scripts/activate
$ pip install -r requirements.txt
```

- migrate 진행

```
$ python manage.py migrate
```

```
Operations to perform:
 Apply all migrations: admin, articles, auth, contenttypes, sessions
Running migrations:
  No migrations to apply.
```

※ migrate 작업 진행 시, 필요 모델이 모두 반영되었는지 반드시 확인!

migrate
모델 구조를 데이터베이스에 반영하는 명령어

<개념9_Django_DRF 1_migrate>

사전 준비 (2/4)

- 사전 제공된 99-json-response-practice 기반 시작
- 준비된 fixtures 파일을 load하여 실습용 초기 데이터 입력

```
$ python manage.py loaddata articles.json
```

```
Installed 20 object(s) from 1 fixture(s)
```

※ fixture 관련 작업 진행 시, 데이터가 모두 올바르게 처리되었는지 반드시 확인!

fixtures
초기 데이터를 자동으로 불러오기 위한 JSON 형식의 데이터 파일

<개념10_Django_DRF 1_fixtures>

사전 준비 (3/4)

- 서버 실행

```
$ python manage.py runserver
```

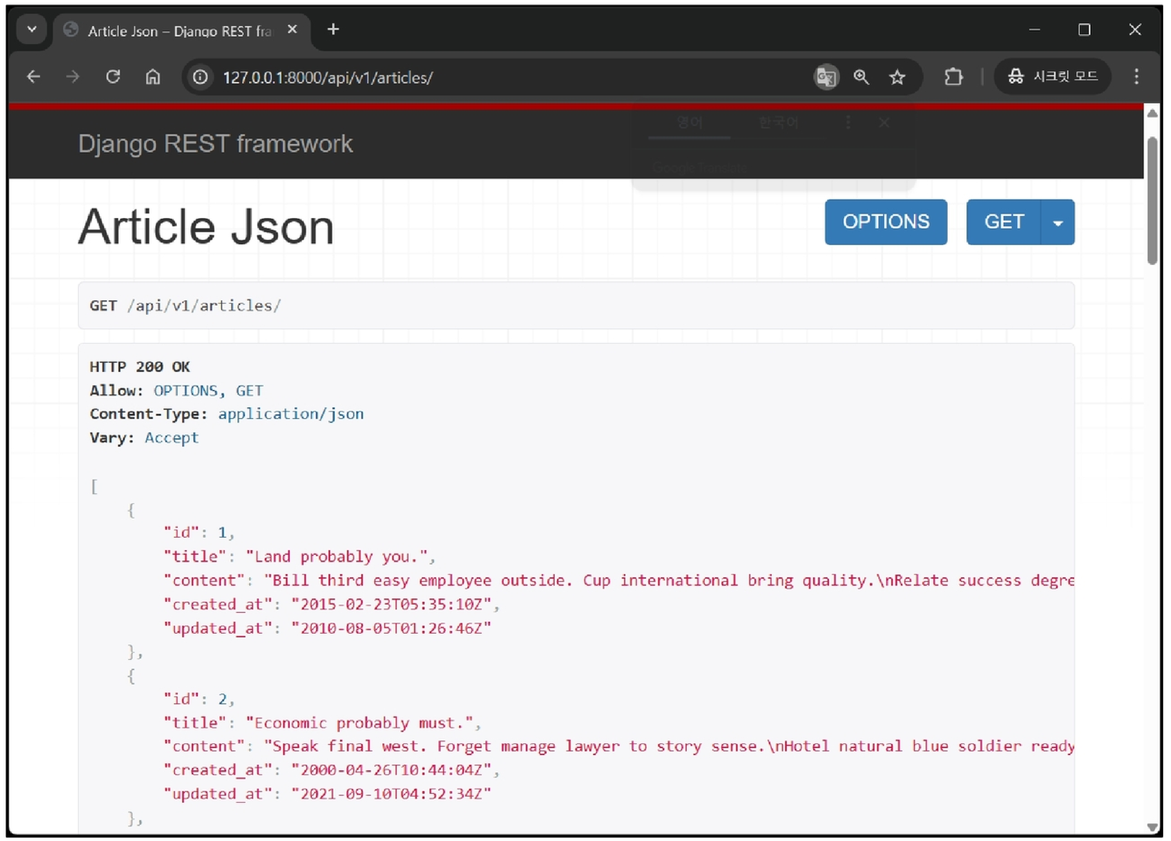
- 터미널 분할 (별도의 파이썬 코드를 실행하기 위함)

<그림17_Django_DRF 1_VSCode 터미널 분할>

사전 준비 (4/4)

- `http://127.0.0.1:8000/api/v1/articles/` 요청 후 응답 확인

```python
# drf/urls.py

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path('api/v1/', include('articles.urls')),
]
```



<그림18_Django_DRF 1_GET 요청 예시 1>

python으로 json 데이터 처리 하기

- 준비된 python-request-sample.py 확인

```
import requests
from pprint import pprint



response = requests.get('http://127.0.0.1:8000/api/v1/articles/')

# json을 python 타입으로 변환
result = response.json()

print(type(result))
pprint(result)
pprint(result[0])
pprint(result[0].get('title'))
```

코드 실행 방법

- 서버가 실행되어 있는 상태에서 2번째 터미널에서 코드 실행

```
$ python python-request-sample.py
```

<그림19_Django_DRF 1_VSCode 터미널 분할 2>

DRF with Single Model

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Django REST Framework

DRF 정의

Django REST framework

Django에서 RESTful API 서버를 쉽게 구축할 수 있도록 도와주는 **오픈소스 라이브러리**

예를 들어, 가구를 직접 만들려면 톱, 망치, 설계도, 재료가 모두 필요합니다.

하지만 **조립식 가구는 드라이버 하나만 있으면 누구나 간단히 완성**할 수 있죠.

Django REST framework는 이와 같이,

복잡한 API 서버 개발 과정을 표준화하고 자동화하여,

초보자도 **빠르고 안정적으로 Restful 구조를 구현할 수 있도록 도와주는 개발 도구 세트** 입니다.

프로젝트 준비

- 사전 제공된 drf 프로젝트 기반 시작
- 가상 환경 생성, 활성화 및 패키지 설치

```
# djangorestframework 설치와 INSTALLED_APPS 등록은 이미 되어있습니다.
$ pip install djangorestframework

# settings.py
INSTALLED_APPS = [
    ...
    'rest_framework',
]
```

- migrate 진행

```
$ python manage.py migrate
```

- 준비된 fixtures 파일을 load하여 실습용 초기 데이터 입력

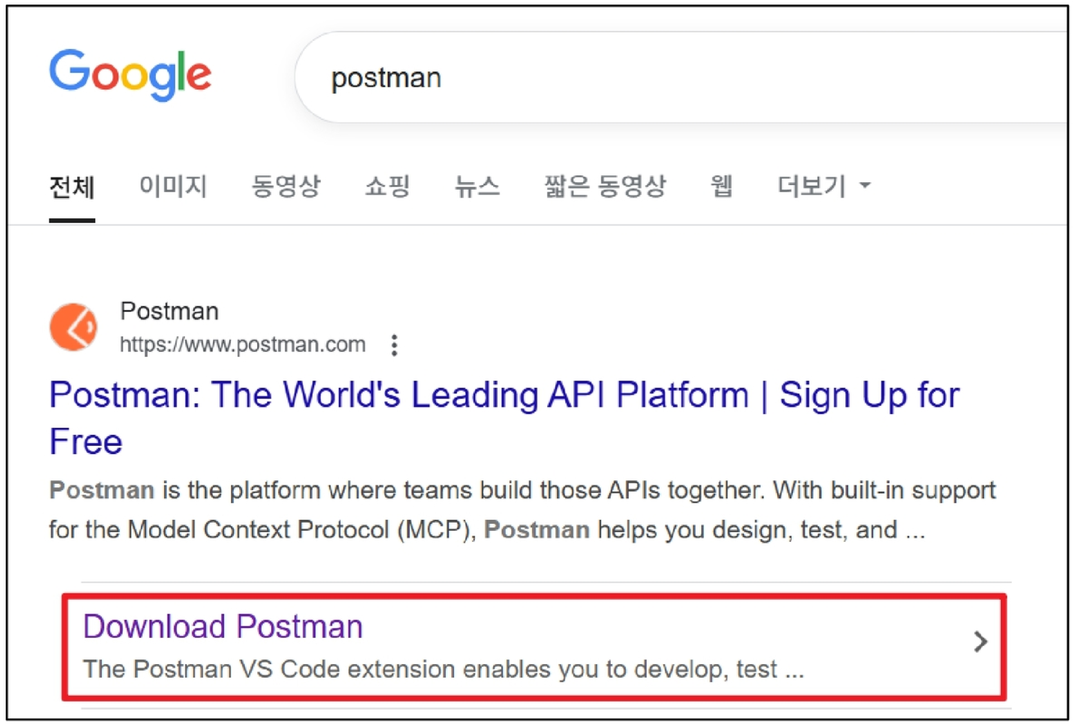
```
$ python manage.py loaddata articles.json
Installed 20 object(s) from 1 fixture(s)
```

fixtures
초기 데이터를 자동으로 불러오기 위한 JSON 형식의 데이터 파일

<개념10_Django_DRF 1_fixtures>

Postman 설치 및 안내 (1/2)

- Postman 설치
  - https://www.postman.com/downloads/

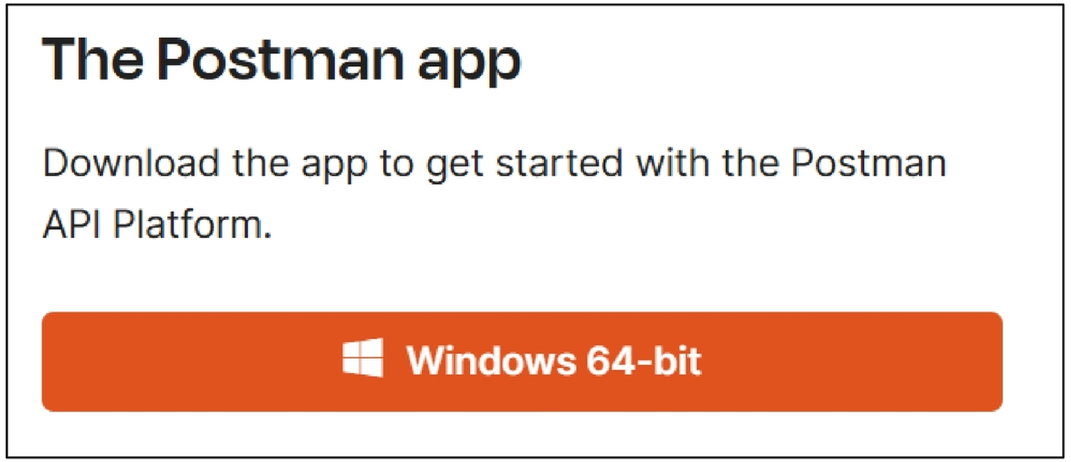


<그림20_Django_DRF 1_postman 검색>

Postman
API 개발 및 테스트를 위한 서비스

요청 데이터 구성, 응답 확인,
환경 설정, 자동화 테스트 등
다양한 기능을 제공

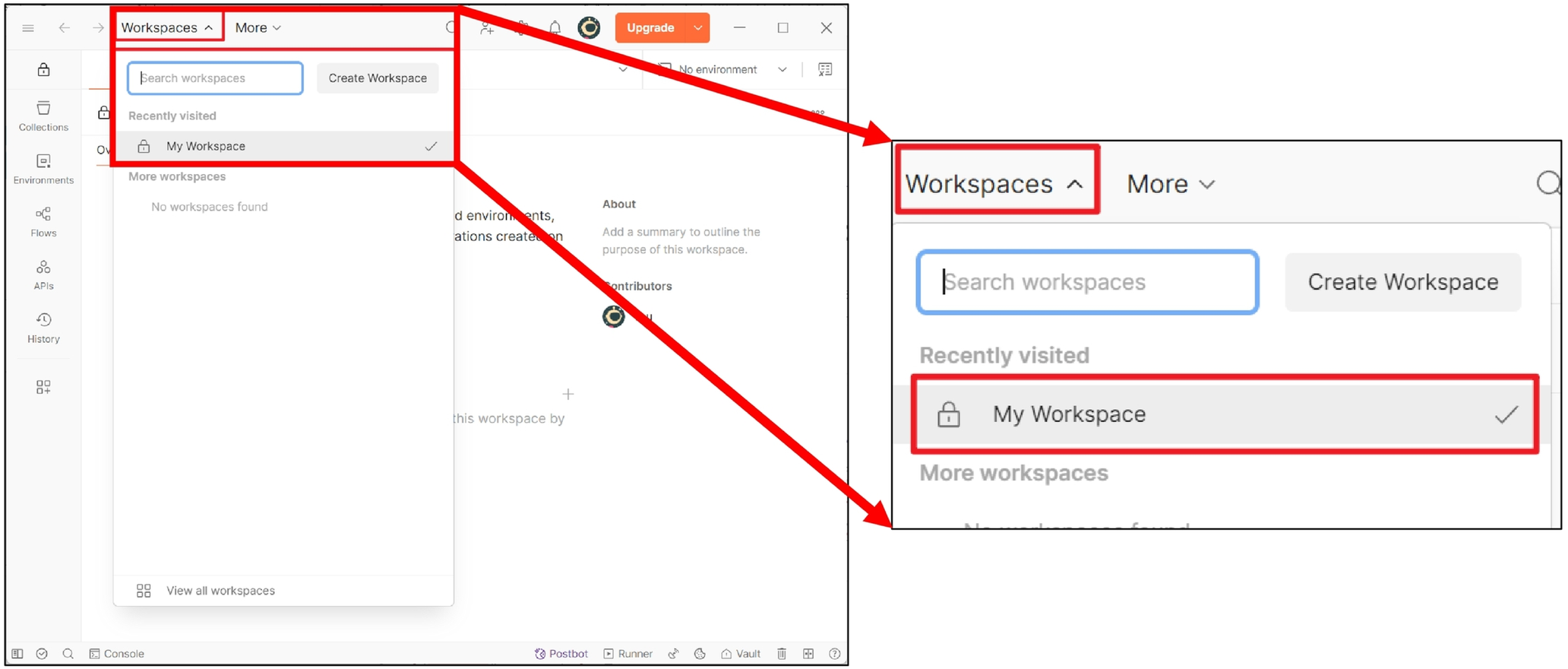
<개념11_Django_DRF 1_Postman>



<그림21_Django_DRF 1_Postman 설치 예시>

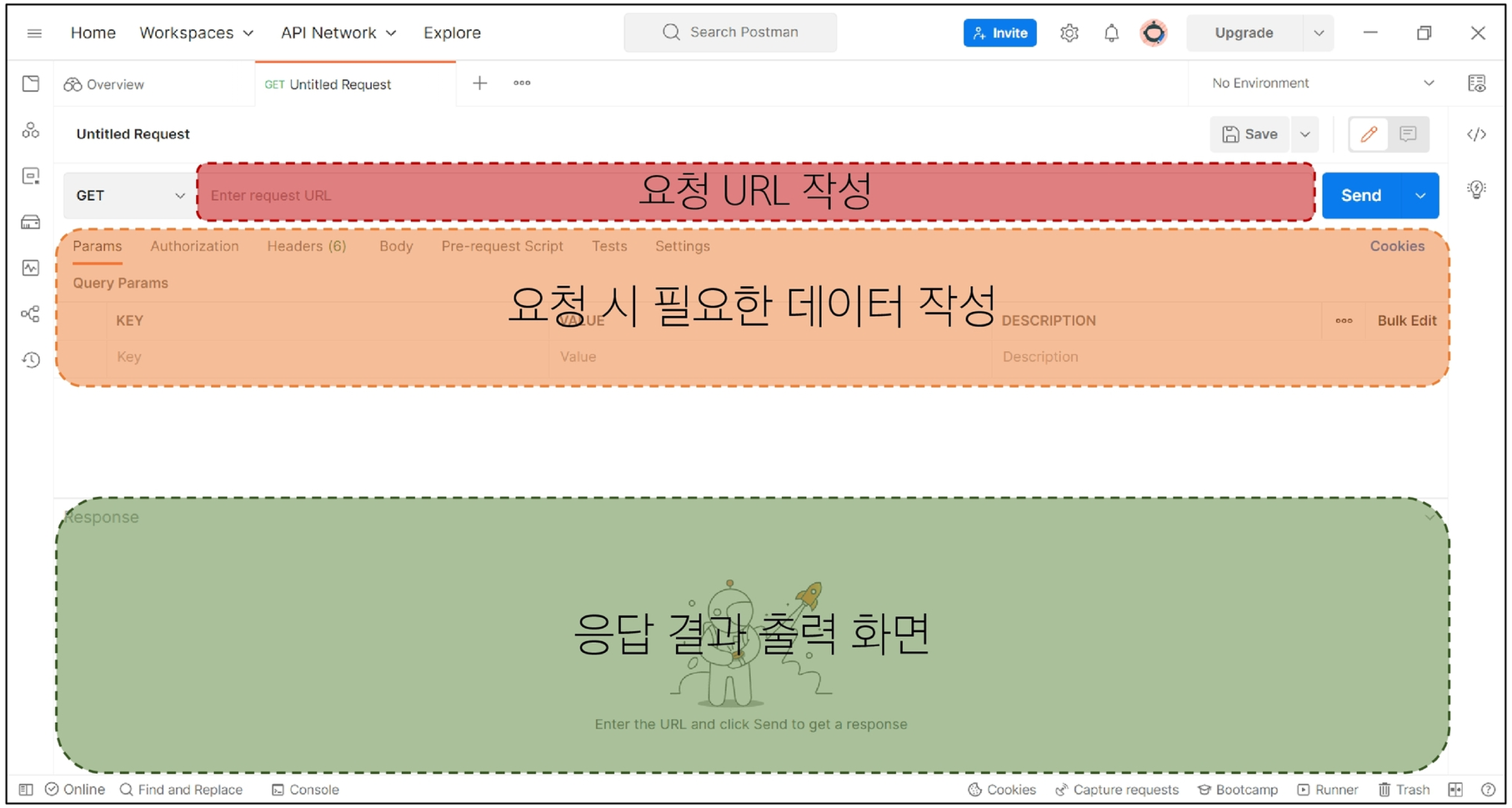
Postman 설치 및 안내 (2/2)

- Workspaces - My workspace



<그림22_Django_DRF 1_Postman 레이아웃 예시>

Postman 화면 구성 안내



<그림23_Django_DRF 1_Postman 화면 구성 안내>

Serializer

Serialization 정의

Serialization

직렬화

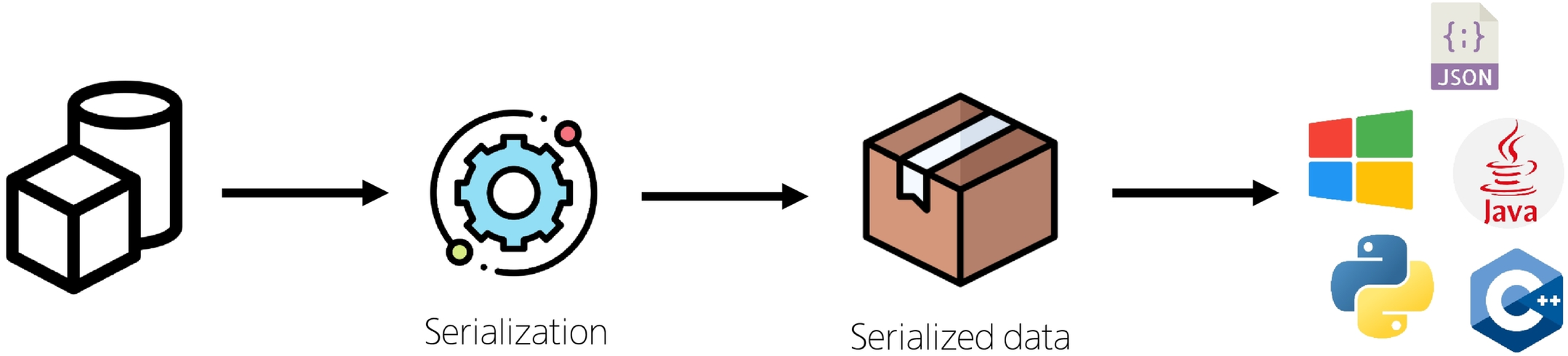
여러 시스템에서 활용하기 위해

데이터 구조나 객체 상태를

재구성할 수 있는 포맷으로 변환하는 과정

Python 객체가 JSON으로 바뀌는 모습

- 데이터 구조나 객체 상태를 나중에 재구성할 수 있는 포맷으로 변환하는 과정



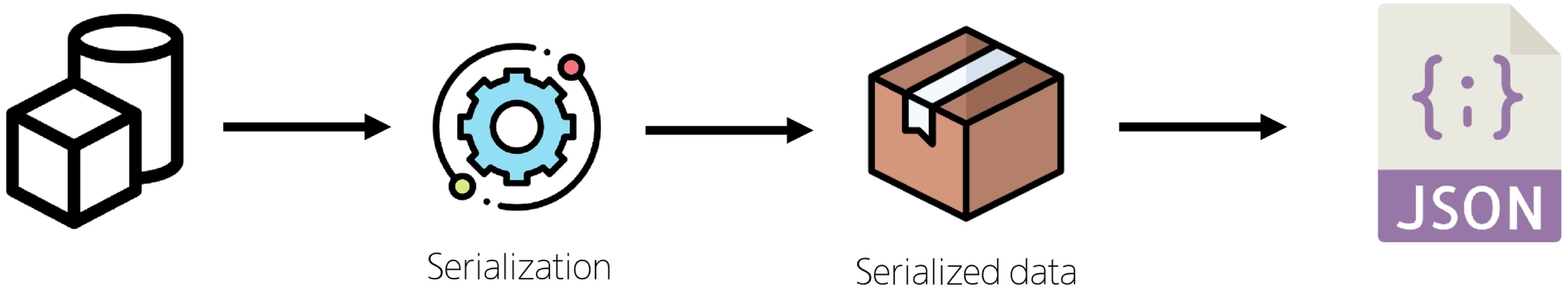
<그림24_Django_DRF 1_Serialization 예시 1>

※ 변환된 데이터는 다른 프로그램, 다른 언어, 다른 컴퓨터에서도 다시 원래의 구조로 복원할 수 있습니다.

※ 즉, 직렬화는 데이터를 “어디서든 읽고 사용할 수 있게 만드는 공통 언어로의 번역”입니다.

수업에서는 JSON 변환에만 집중

- 데이터 구조나 객체 상태를 나중에 재구성할 수 있는 포맷으로 변환하는 과정



<그림25_Django_DRF 1_Serialization 예시 2>

수업에서는 JSON으로의 변환에만 집중합시다.

Serializer 정의

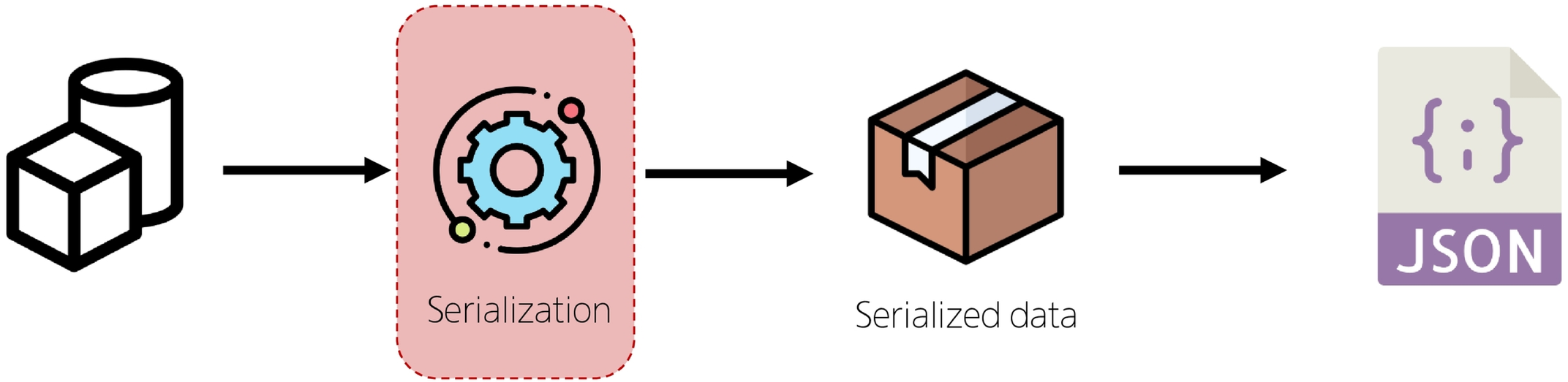
Serializer

Serialization(직렬화)을 수행하는 도구(클래스)

구분	Serialization	Serializer
정체	과정 (동작)	도구 (클래스)
비유	번역	번역가
코드에서	`serializer.data`를 꺼내는 순간 일어나는 일	`serializers.py`에 작성하는 클래스

Django 프로젝트에서 Serializer가 맡는 구간

- 데이터 구조나 객체 상태를 나중에 재구성할 수 있는 포맷으로 변환하는 과정



Serializer Class 담당

<그림26_Django_DRF 1_Serialization 예시 3>

serializers.py 파일을 생성하여 작성

ModelSerializer

- Django 모델과 연결된 Serializer 클래스

➢ 모델 정보를 바탕으로 필드, 유효성 검사, 저장(create/update) 기능을 자동으로 만들어 주는 Serializer (Form과 ModelForm의 관계와 같음)

TIP

Serializer는 단순히 포맷 변환 도구가 아닌, 값 검증, 데이터 구조 정의, 모델 연동까지 담당하는 핵심 계층

폼을 사용하는 Django 웹 개발과 동일하게, API 기반 개발에서는 Serializer가 데이터 입출력의 중심

ModelSerializer class 구조 소개

- Article 모델을 토대로 직렬화를 수행하는 ArticleSerializer 정의

※ 실제 작성은 이어지는 CRUD 실습에서 진행

```python
# articles/forms.py (Django Form 수업)

class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = '__all__'
```

```python
# articles/serializers.py (오늘)

class ArticleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = '__all__'
```

TIP

Form은 HTML 화면용, Serializer는 JSON 데이터용. 모양도 하는 일(필드 정의, 유효성 검사, 저장)도 거의 같습니다. Meta의 fields가 곧 "응답 JSON의 모양"을 결정합니다.

CRUD with ModelSerializer

URL과 HTTP request methods 설계

※ 오늘 진행 할 프로젝트의 구성입니다.

	GET	POST	PUT	DELETE
articles/	전체 글 조회	글 작성	.	.
articles/1/	1번 글 조회	.	1번 글 수정	1번 글 삭제

<표1_Django_DRF 1_프로젝트 HTTP requests method 설계도>

TIP

- URL에 동작명(get, create)을 넣지 말고, 자원 중심으로 설계하세요.
- 복수형/단수형 혼용은 혼란을 주니 일관되게 사용하세요.
- 깊은 중첩 구조는 피하고, 필요한 경우 관계를 명확히 표현하세요.
- 기능이 아닌 자원의 위치만 URL로 표현하고, 동작은 HTTP 메서드로 구분하세요.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

GET method - 조회

GET - List (1/3)

- 게시글 데이터 목록 조회하기
- 게시글 데이터 목록을 제공하는 ArticleListSerializer 정의

```python
# articles/serializers.py

from rest_framework import serializers
from .models import Article

class ArticleListSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = ('id','title','content',)
```

ModelSerializer
Django 모델 구조를 바탕으로 자동으로 필드를 생성해주는 Serializer 클래스

<개념12_Django_DRF 1_ModelSerializer>

왜 Serializer를 하나 더 만들까?

"목록 화면과 상세 화면은 보여줄 정보가 다르다"

- 쇼핑몰을 떠올려 보세요. 상품 목록에는 이름과 가격만, 상품 상세에는 모든 정보가 나옵니다.
- API도 같습니다. 용도마다 fields가 다른 Serializer를 따로 만듭니다.

용도	Serializer	fields	URL
목록 조회	`ArticleListSerializer`	`id, title, content`	`articles/`
상세 조회·생성·수정	`ArticleSerializer`	`'__all__'`	`articles/1/`

GET - List (2/3)

- url 및 view 함수 작성

```python
# articles/views.py

from rest_framework.response import Response
from rest_framework.decorators import api_view

from .models import Article
from .serializers import ArticleListSerializer


@api_view(['GET'])
def article_list(request):
    articles = Article.objects.all()
    serializer = ArticleListSerializer(articles, many=True)
    return Response(serializer.data)
```

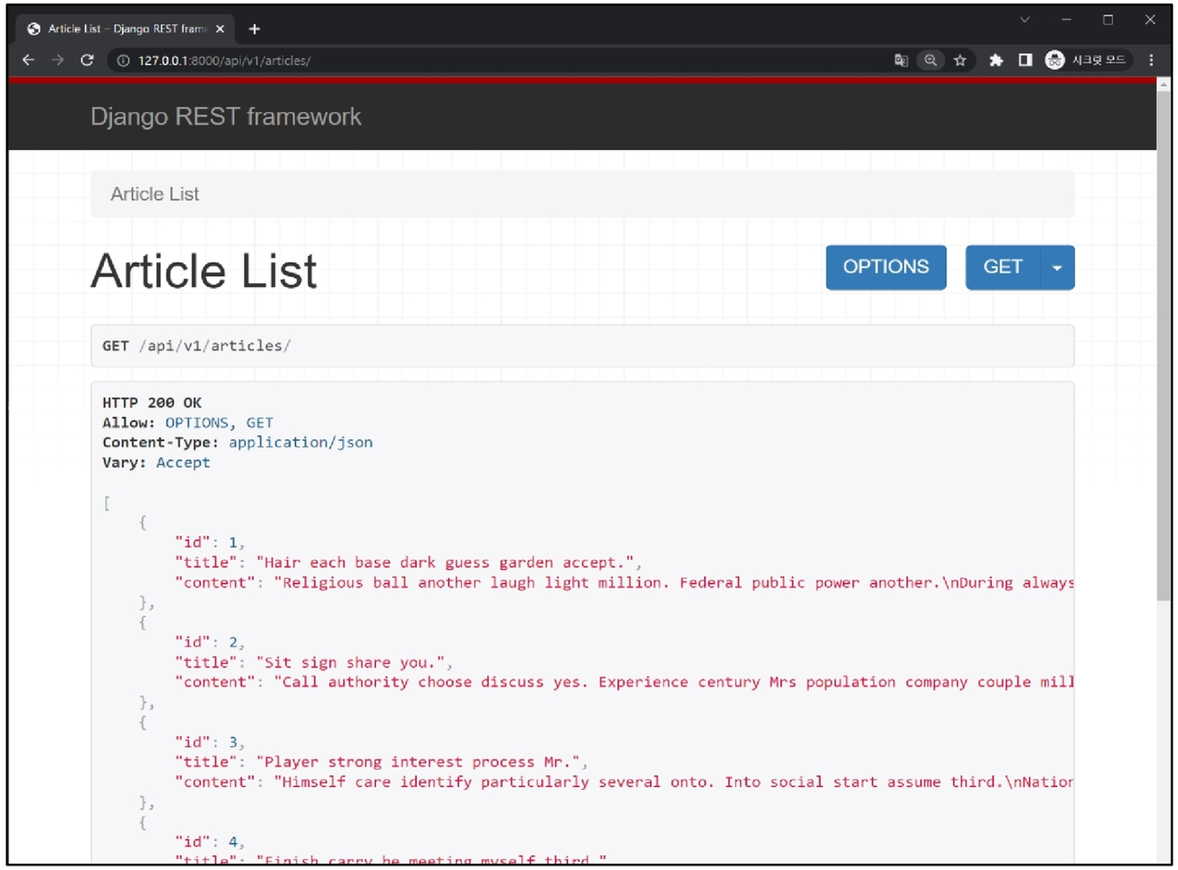
@api_view
함수형 뷰에서 사용할
HTTP 메서드를 명시해주는
DRF 전용 데코레이터

<개념13_Django_DRF 1_api view 데코레이터>

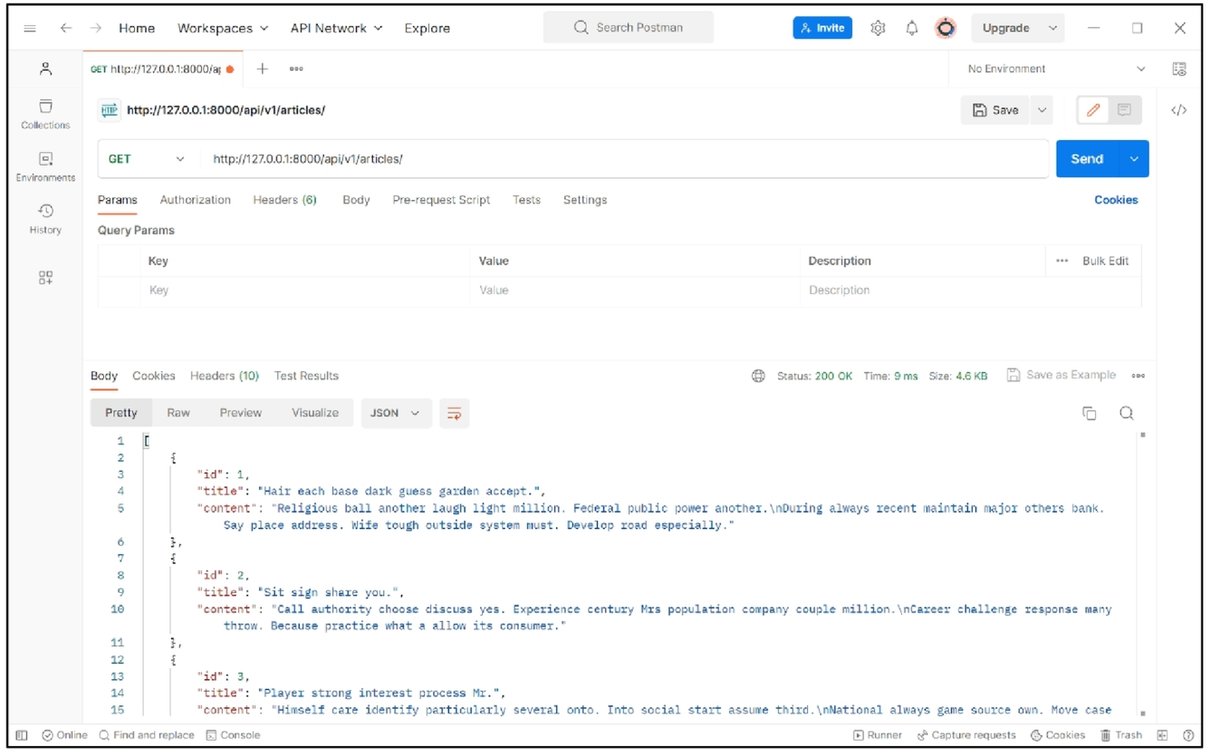
```python
# articles/urls.py
urlpatterns = [
    path('articles/', views.article_list),
    ...
]
```

GET - List (3/3)

- http://127.0.0.1:8000/api/v1/articles/ 응답 확인



<그림27_Django_DRF 1_게시글 전체 조회 화면>



<그림28_Django_DRF 1_게시글 전체 조회 Postman>

ModelSerializer의 인자 및 속성

- many 옵션
  - Serialize 대상이 QuerySet인 경우 입력

```
serializer = ArticleListSerializer(articles, many=True)
```

※ many 옵션을 지정하지 않으면 단일 객체로 처리됩니다.

- data 속성
  - serializer 객체에서 직렬화된 결과 데이터를 꺼냄

```
return Response(serializer.data)
```

QuerySet
Django ORM을 통해 모델에서 조회된 객체들의 집합

반복문으로 순회하거나 필터링 등 다양한 연산이 가능함

<개념14_Django_DRF 1_QuerySet>

과거 view 함수와의 응답 데이터 비교

- 똑같은 데이터를

  과거) HTML에 출력되도록 페이지와 함께 응답했던 view 함수,

```python
def index(request):
    articles = Article.objects.all()
    context = {
        'articles': articles,
    }
    return render(request, 'articles/index.html', context)
```

현재) JSON 데이터로 serialization 하여 페이지 없이 응답하는 view 함수

```python
@api_view(['GET'])
def article_list(request):
    articles = Article.objects.all()
    serializer = ArticleListSerializer(articles, many=True)
    return Response(serializer.data)
```

‘api_view’ decorator

- DRF view 함수에서는 **필수로 작성**되며 view 함수를 실행하기 전 HTTP 메서드를 확인
- 허용하도록 지시한 메서드에 대해서만 올바르게 답변하며,
  목록에 추가하지 않은 다른 메서드 요청에 대해서는 **405 Method Not Allowed**로 응답
- DRF view 함수가 응답해야 하는 HTTP 메서드 목록을 작성

TIP

@api_view 데코레이터를 빠뜨리면,
함수가 단순한 Django 뷰로 인식되어 API 요청이 제대로 처리되지 않습니다.
➔ DRF에서는 반드시 @api_view([...])를 작성해야 하고,
생략할 경우 500번 에러나 HTML 응답이 반환되는 등 **원인을 찾기 어려운 오류**가 발생할 수 있습니다.
요청이 실패할 땐 데코레이터 누락 여부부터 확인하세요.

GET - Detail (1/3)

- 단일 게시글 데이터 조회하기
  - 각 게시글의 상세 정보를 제공하는 `ArticleSerializer` 정의

```python
# articles/serializers.py

from rest_framework import serializers
from .models import Article


class ArticleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = '__all__'
```

ModelSerializer
Django 모델 구조를 바탕으로 자동으로 필드를 생성해주는 Serializer 클래스

<개념12_Django_DRF 1_ModelSerializer>

GET - Detail (2/3)

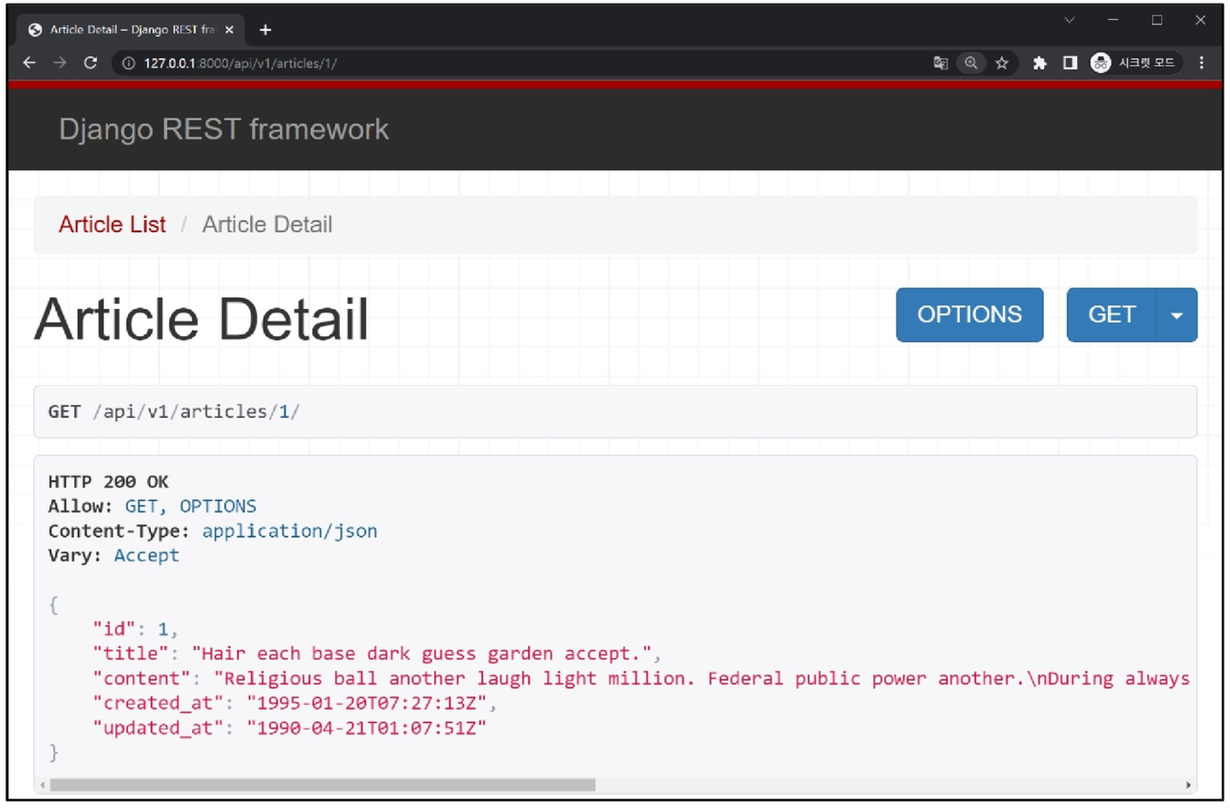
- url 및 view 함수 작성

```
# articles/urls.py

urlpatterns = [
    ...
    path('articles/<int:article_pk>/', views.article_detail),
]
```

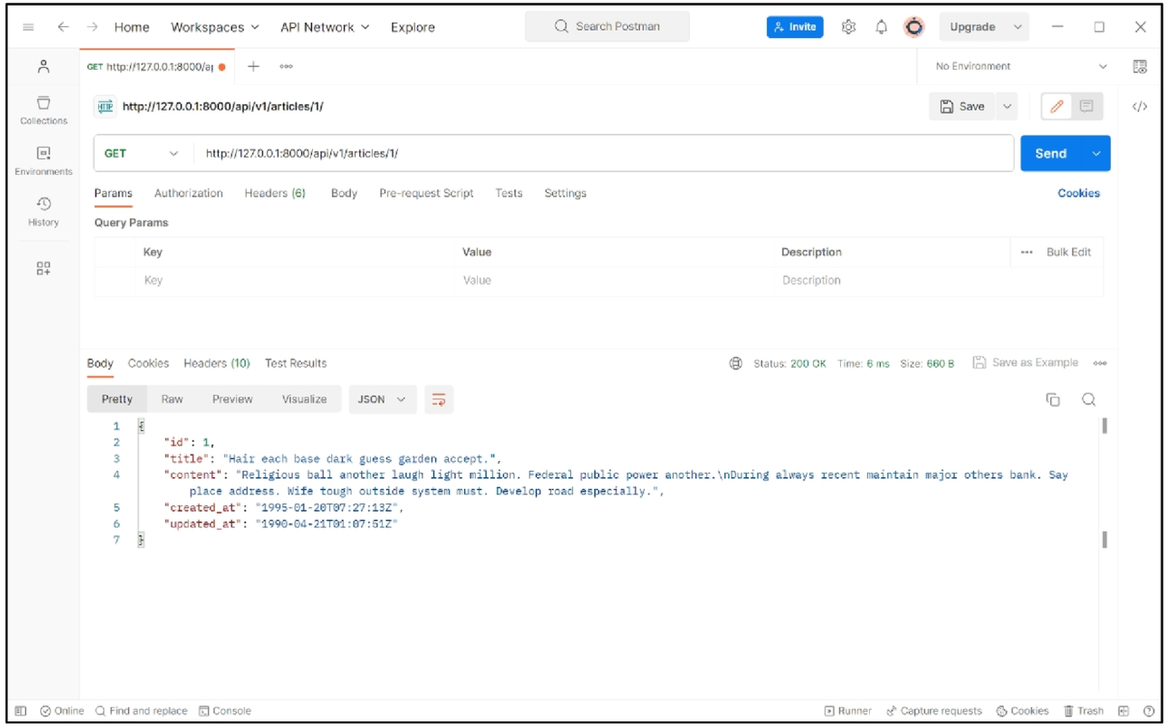
```
# articles/views.py
from .serializers import ArticleListSerializer, ArticleSerializer

@api_view(['GET'])
def article_detail(request, article_pk):
    article = Article.objects.get(pk=article_pk)
    serializer = ArticleSerializer(article)
    return Response(serializer.data)
```

GET - Detail (3/3)

- http://127.0.0.1:8000/api/v1/articles/1/ 응답 확인



<그림29_Django_DRF 1_게시글 상세 조회 화면>



<그림30_Django_DRF 1_게시글 상세 조회 Postman>

POST method - 생성

POST (1/4)

- 게시글 데이터 생성하기

1. 데이터 생성이 성공했을 경우 201 Created 응답
2. 데이터 생성이 실패 했을 경우 400 Bad request 응답

TIP

POST 요청에서 에러가 발생할 경우, Serializer의 is_valid()를 먼저 확인하세요.
→ 어떤 필드가 누락되었는지, 형식이 잘못되었는지 정확한 힌트를 얻을 수 있습니다.
400 오류는 대부분 입력 데이터 문제입니다.

POST (2/4)

- `article_list view` 함수 구조 변경 (method에 따른 분기처리)
- 생성할 때는 입력받을 필드를 검사하고, 저장된 결과를 전체 정보로 돌려주어야 하므로 `ArticleSerializer` 사용

```python
# articles/views.py
from rest_framework import status

@api_view(['GET', 'POST'])
def article_list(request):
    if request.method == 'GET':
        articles = Article.objects.all()
        serializer = ArticleListSerializer(articles, many=True)
        return Response(serializer.data)

    elif request.method == 'POST':
        serializer = ArticleSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

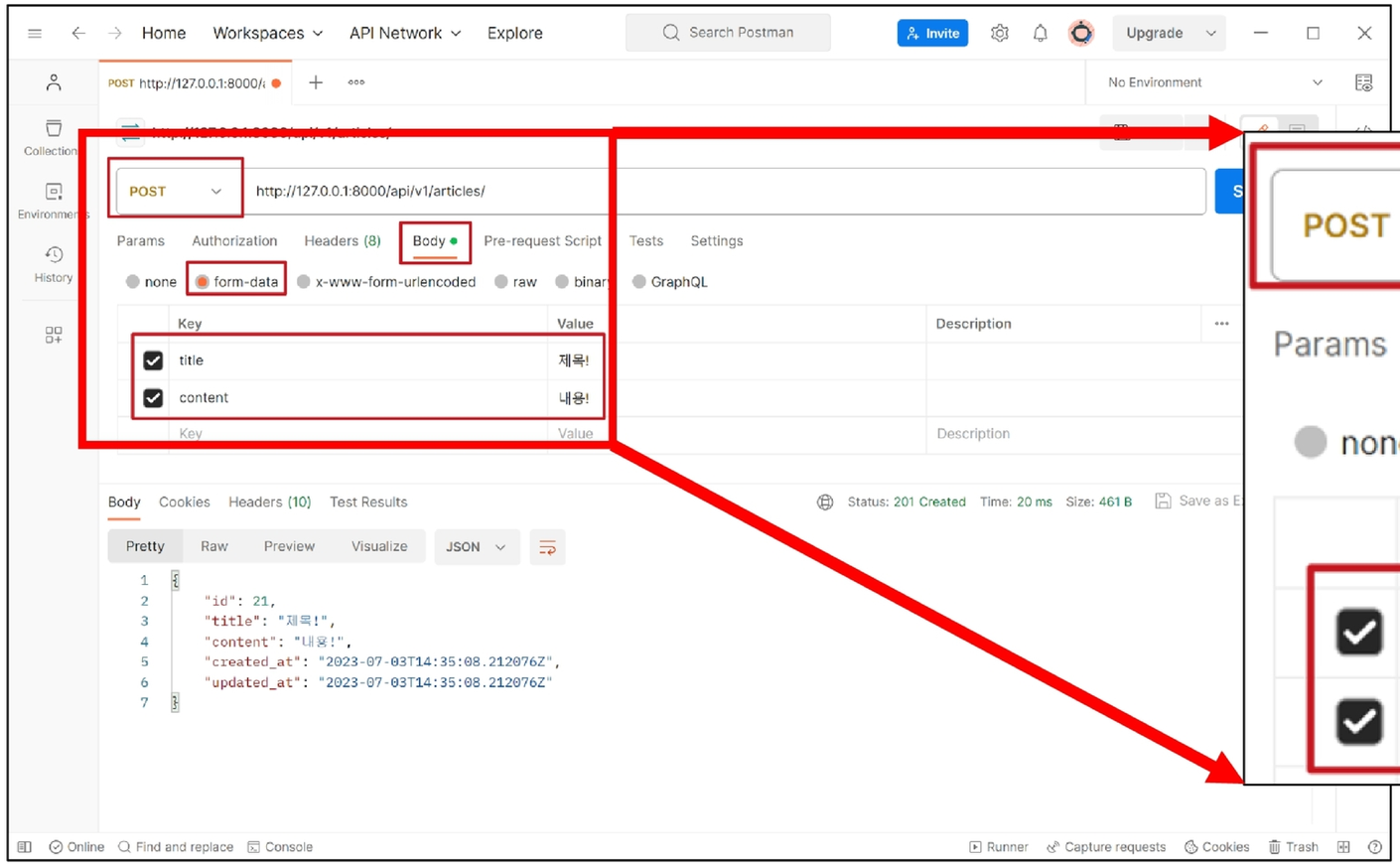
※ 리스트 형태로 허용할 HTTP 메서드들을 지정할 수 있음

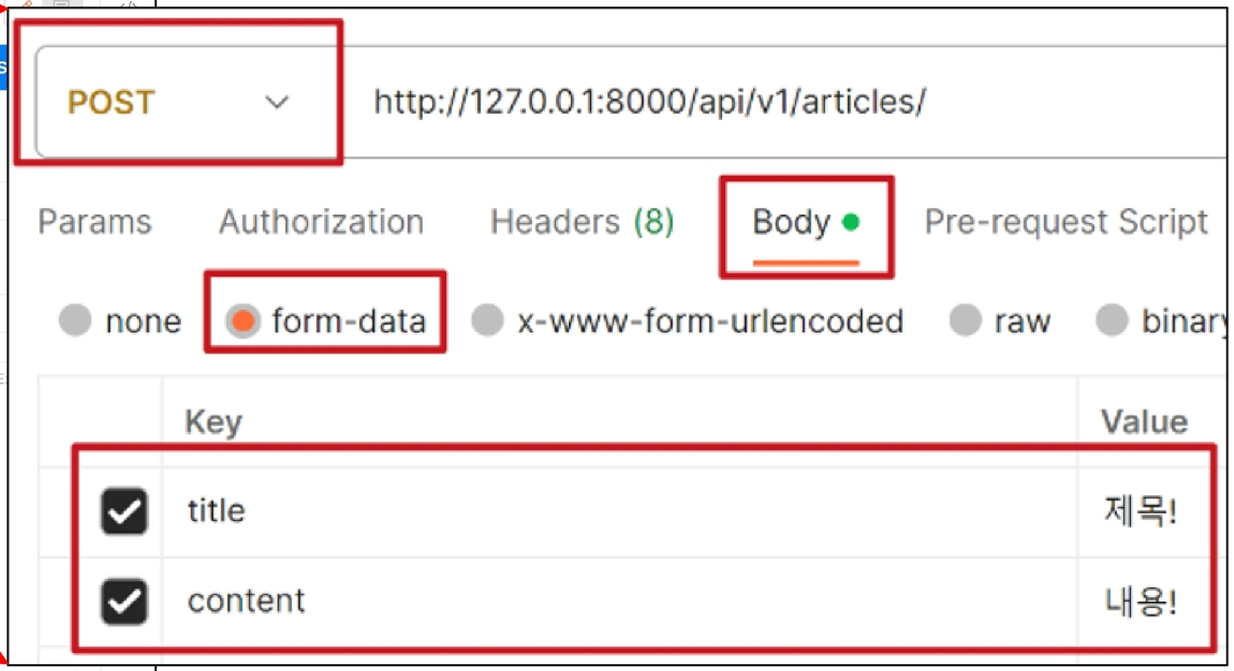
POST (3/4)

- POST http://127.0.0.1:8000/api/v1/articles/ 응답 확인

form-data
폼 형식으로 전송되는 요청 본문 데이터로, 파일 업로드나 다중 필드 전송에 사용됨

<개념15_Django_DRF 1_form-data>

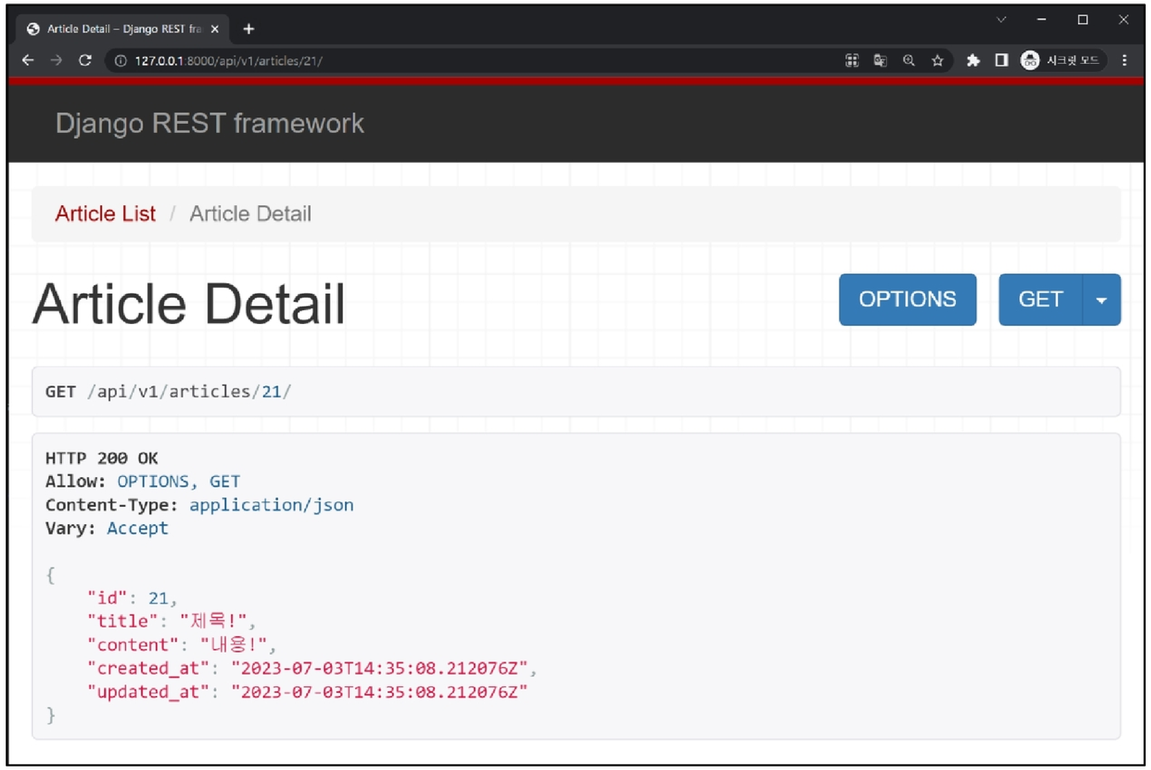




<그림31_Django_DRF 1_게시글 생성 요청 Postman>

POST (4/4)

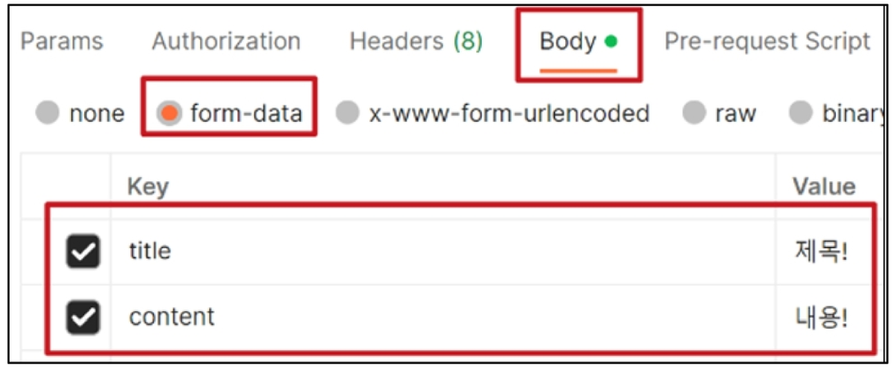
- 새로 생성된 게시글 데이터 확인
  - GET http://127.0.0.1:8000/api/v1/articles/{{ article_pk }}/



<그림29_Django_DRF 1_게시글 상세 조회 화면>

※ article pk는 각자 상황마다 다를 수 있음

※ 입력한 데이터와 일치하는지 확인하는 데에 집중!



<그림32_Django_DRF 1_POST 요청 form-data>

DELETE method - 삭제

DELETE (1/2)

- 게시글 데이터 삭제하기
  - 요청에 대한 데이터 삭제가 성공했을 경우는 **204 No Content** 응답

204 No Content
요청은 성공했지만,
응답으로 보낼 본문 데이터가
없음을 나타내는 상태 코드

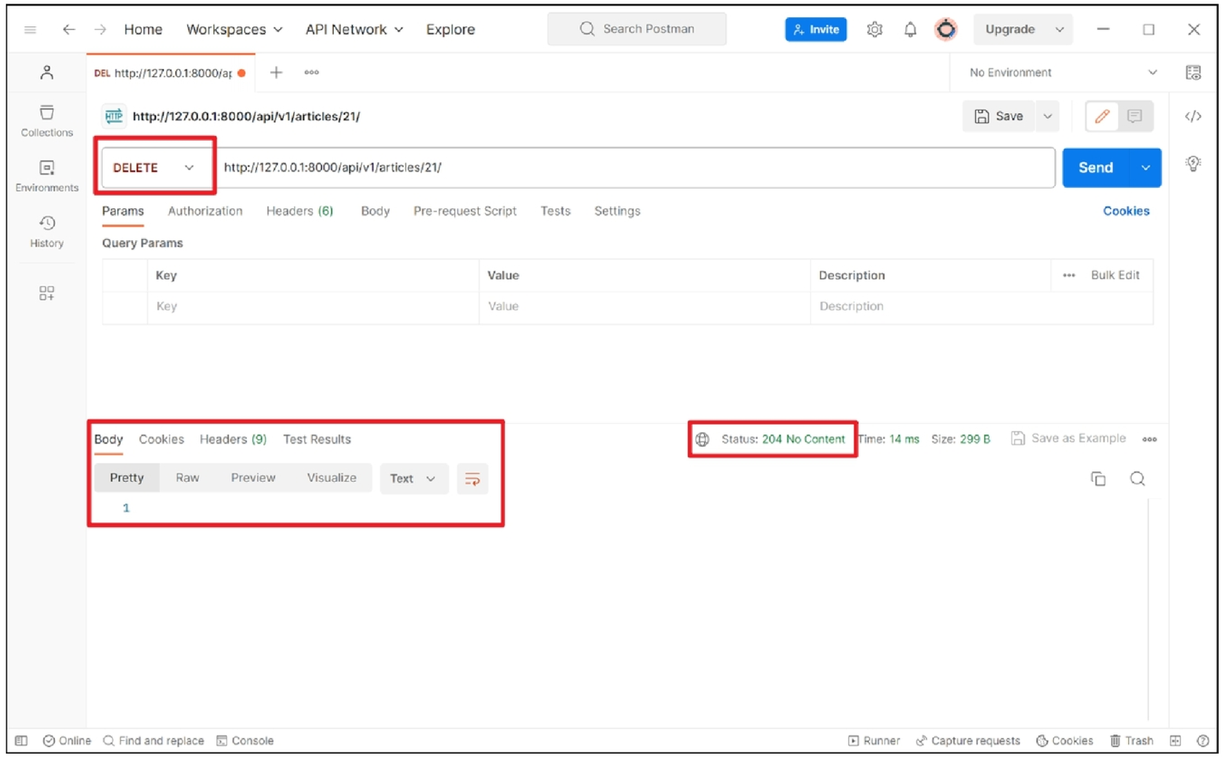
<개념16_Django_DRF 1_204 No Content>

```python
# articles/views.py

@api_view(['GET', 'DELETE'])
def article_detail(request, article_pk):
    article = Article.objects.get(pk=article_pk)
    if request.method == 'GET':
        serializer = ArticleSerializer(article)
        return Response(serializer.data)

    elif request.method == 'DELETE':
        article.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

DELETE (2/2)

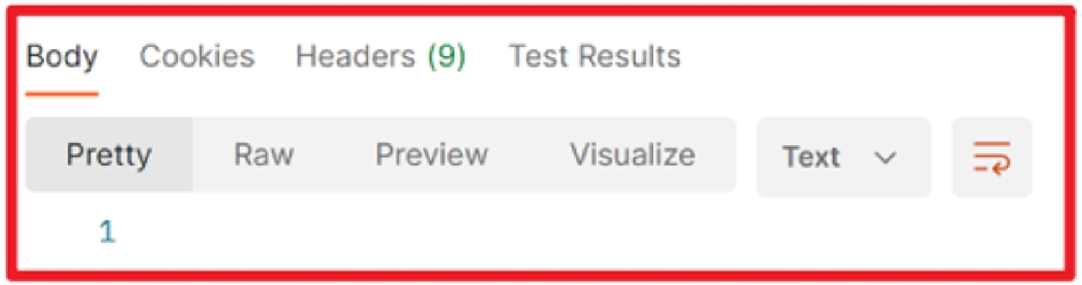
- DELETE http://127.0.0.1:8000/api/v1/articles/{{ article_pk }}/ 응답 확인

article pk는 각자 상황마다 다를 수 있음



<그림33_Django_DRF 1_게시글 삭제 요청 Postman>

서버가 반환하는 별도의 데이터 없음



<그림34_Django_DRF 1_게시글 삭제 요청 응답 Postman>

DELETE 응답 시 Response 구성 방식

- Response()는 기본적으로 data 인자를 필요로 하지 않음
- **아무런 데이터도 넘기지 않을 경우엔 응답 상태 코드만으로 결과를 전달**
  - 첫번째 인자를 비운 상태로 두번째 인자에 값을 전달할 수 없으므로, 키워드 인자형태로 값 전달

```python
# articles/views.py

@api_view(['GET', 'DELETE'])
def article_detail(request, article_pk):
    ...

    elif request.method == 'DELETE':
        article.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

```python
class Response(
    data: Any | None = None,
    status: Any | None = None,
    template_name: Any | None = None,
    headers: Any | None = None,
    exception: bool = False,
    content_type: Any | None = None
)
```

<그림35_Django_DRF 1_Response 클래스 구성>

DELETE 응답 시 데이터를 반환하는 방법

- 일반적으로 DELETE 요청은 204 No Content로 본문 없이 응답하는 것이 RESTful 한 설계방식
- 하지만 특정 상황에서는 **삭제된 객체의 정보를 함께 반환**해야 할 수도 있음
  - 클라이언트에서 **삭제 대상 데이터를 확인**하거나, **UI에서 알림 메시지로 활용**할 경우
    - ➢ 예: "게시글 'Django 소개'가 삭제되었습니다."
  - 삭제된 데이터가 실제로 무엇이었는지 **사용자에게 피드백을 주기 위해**

TIP

- **DELETE에서도 응답이 필요한 경우엔 204 대신 200 OK 상태 코드와 함께 데이터를 반환하세요.**
- **단, REST 원칙상 기본은 응답 없음(204) 이므로, 목적이 명확할 때만 사용합니다.**

DELETE 처리 후, 추가 응답 데이터 반환 (1/2)

- 게시글 데이터를 삭제하고, 삭제된 게시글 정보 반환하기
  - 추가적인 데이터를 제공하므로 **200 OK** 응답

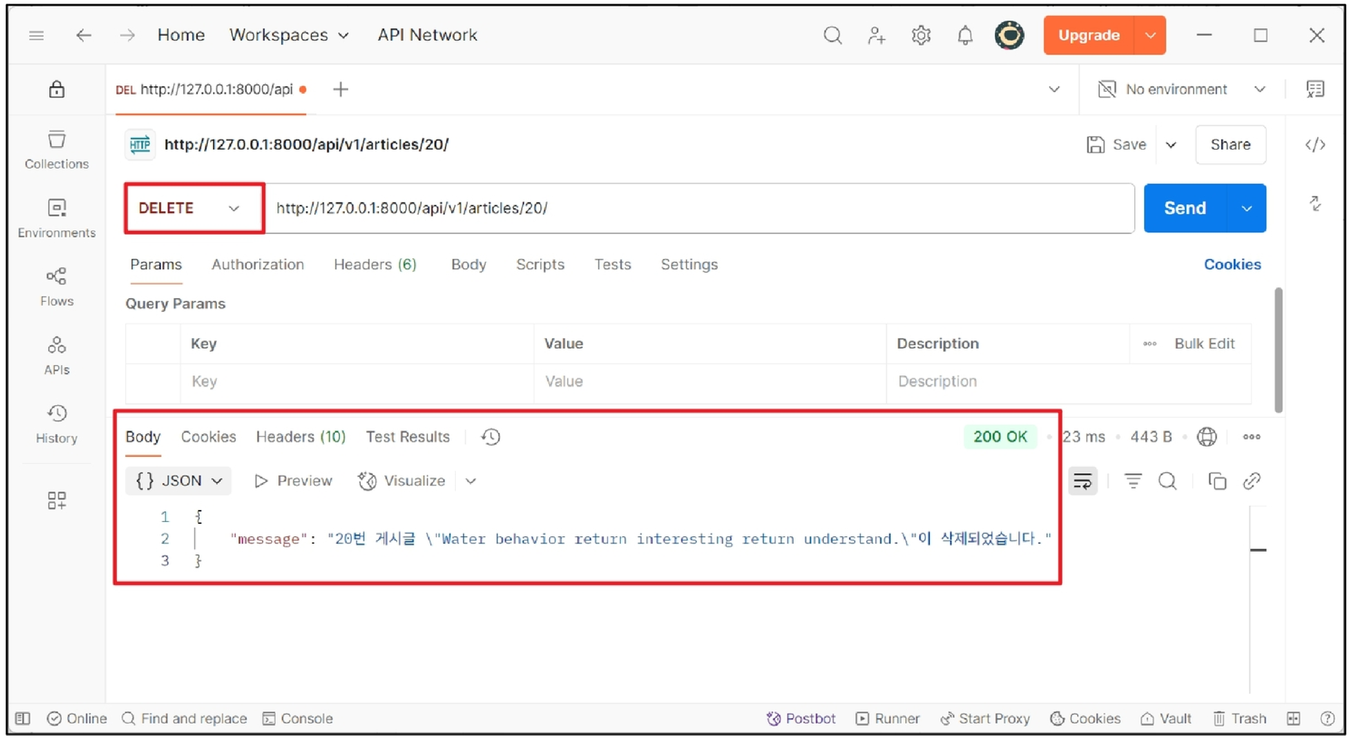
1. 반환할 데이터를 정의
   - `delete()` 실행 시, 해당 객체는 데이터베이스에서 삭제 됨
   - 그러므로 필요한 값은 삭제 전에 미리 변수로 저장해두고,
   - 삭제 이후에는 변수만 사용하는 것이 더 안정적
2. Response의 첫번째 인자로 전달

```python
# articles/views.py

@api_view(['GET', 'DELETE'])
def article_detail(request, article_pk):
    article = Article.objects.get(pk=article_pk)
    ...
    elif request.method == 'DELETE':
        pk = article.pk
        title = article.title
        article.delete()
        data = {
            'message': f'{pk}번 게시글 "{title}"이 삭제되었습니다.'
        }
        return Response(data, status=status.HTTP_200_OK)
```

DELETE 처리 후, 추가 응답 데이터 반환 (2/2)

- DELETE http://127.0.0.1:8000/api/v1/articles/{{ article_pk }}/ 응답 확인

※ article pk는 각자 상황마다 다를 수 있음



<그림36_Django_DRF 1_게시글 삭제 요청 Postman 데이터>

PUT method - 수정

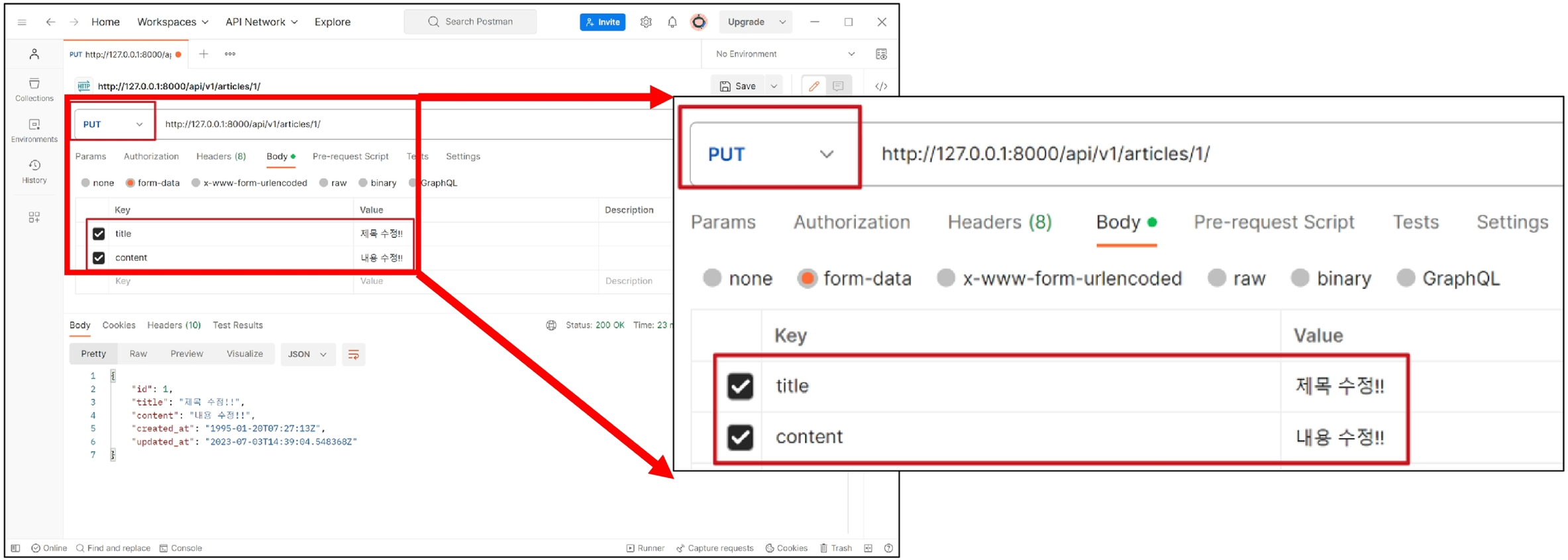
PUT (1/3)

- 게시글 데이터 수정하기
  - 요청에 대한 데이터 수정이 성공했을 경우는 200 OK 응답

```python
# articles/views.py

@api_view(['GET', 'DELETE', 'PUT'])
def article_detail(request, article_pk):
    ...
    elif request.method == 'PUT':
        serializer = ArticleSerializer(article, data=request.data)
        # serializer = ArticleSerializer(instance=article, data=request.data, partial=False)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

PUT (2/3)

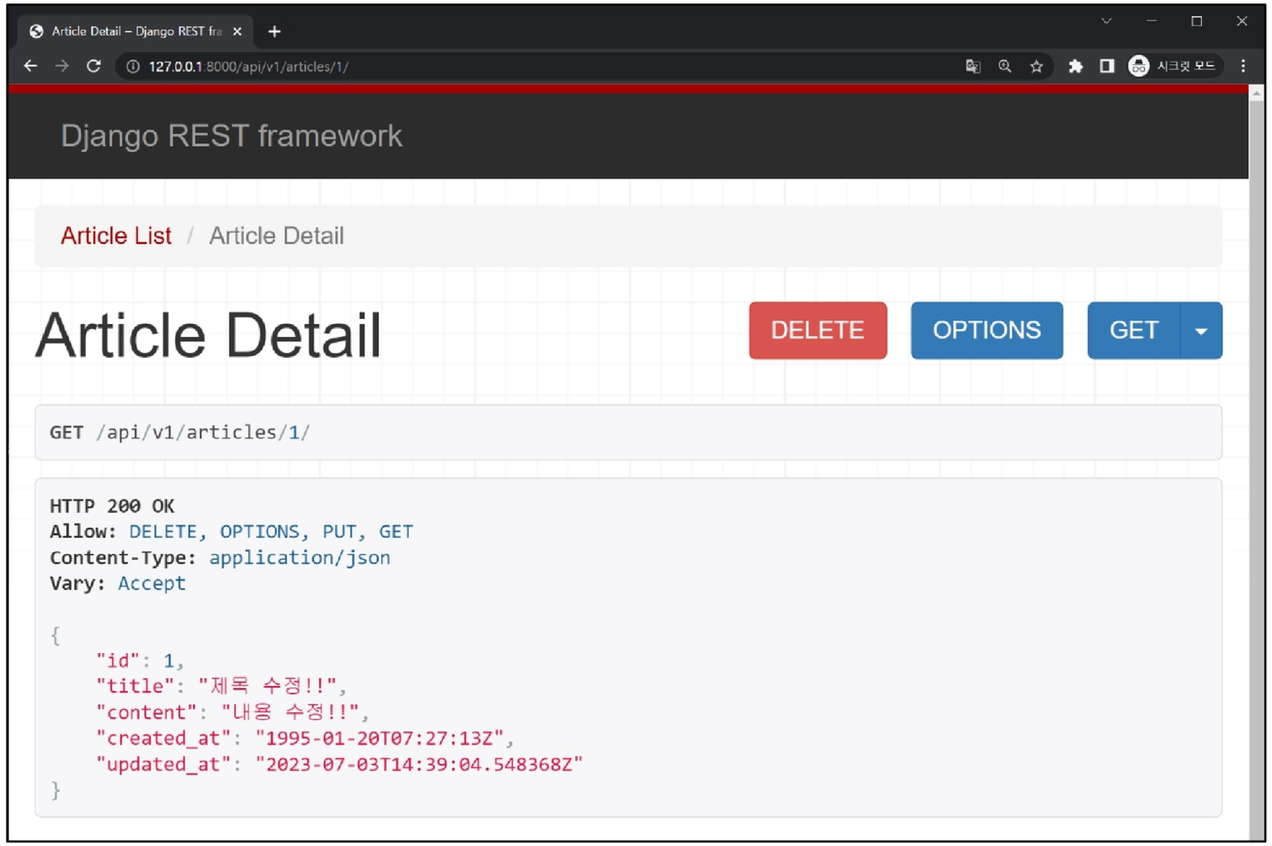
- PUT http://127.0.0.1:8000/api/v1/articles/1/ 응답 확인



<그림37_Django_DRF 1_게시글 수정 요청 Postman>

PUT (3/3)

- GET http://127.0.0.1:8000/api/v1/articles/1/ 수정된 데이터 확인



<그림38_Django_DRF 1_게시글 수정 후 상세 조회 화면>


```
{
    "id": 1,
    "title": "제목 수정!!",
    "content": "내용 수정!!",
    "created_at": "1995-01-20T07:27:13Z",
    "updated_at": "2023-07-03T14:39:04.548368Z"
}
```


<그림39_Django_DRF 1_게시글 수정 요청 후 반환 데이터>

‘partial’ argument

- “부분 업데이트”를 허용하기 위한 인자
- **`partial` 인자의 기본 값은 `False`**
- `partial` 인자를 `False`로 설정하면, 게시글의 title만 수정하려고 하더라도 content와 같은 **다른 필수 필드들도 함께 전송해야 함**
- 이는 serializer가 **기본적으로 모든 필수 필드에 대한 값이 전달되었는지 확인하기 때문**
- 따라서 **일부 필드만 수정**하고 싶다면, **`partial=True`**로 설정하여 일부 필드만 전달되도록 허용해야 함

PATCH 메서드 - 일부 필드만 수정하기

- PATCH는 리소스의 전체가 아닌, 일부만 수정할 때 사용하는 HTTP 메서드
- Django REST framework에서는 `partial=True` 설정을 통해 부분 수정을 구현
  - 게시글의 title만 바꾸고 싶을 때, 전체 필드를 다 보낼 필요 없이 해당 필드만 전송하면 됨

```python
# articles/views.py

@api_view(['GET', 'DELETE', 'PATCH'])
def article_detail(request, article_pk):
    ...
    elif request.method == 'PATCH':
        serializer = ArticleSerializer(article, data=request.data, partial=True)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

PUT vs PATCH

항목	PUT	PATCH
수정 대상	전체 리소스	리소스의 일부 필드
요청 데이터 요구	모든 필수 필드 포함	수정할 필드만 포함 가능
사용 목적	전체 덮어쓰기 (교체)	부분 수정 (일부 필드만 갱신)
DRF 설정	기본 (partial=False)	반드시 partial=True 필요

<표2_Django_DRF 1_PUT과 PATCH 비교>

TIP

- 일부 필드만 수정할 땐 반드시 PATCH를 사용해야 RESTful한 설계입니다.
- PUT 요청에서 partial=True를 사용하는 것은 편의상 허용되기도 하지만, REST 원칙에는 어긋납니다.

참고

raise_exception

raise_exception

- is_valid()의 선택 인자
- 유효성 검사를 통과하지 못할 경우 ValidationError 예외를 발생시킴
- DRF에서 제공하는 기본 예외 처리기에 의해 자동으로 처리되며 기본적으로 HTTP 400 응답을 반환

```python
# articles/views.py

@api_view(['GET', 'POST'])
def article_list(request):
    ...
    elif request.method == 'POST':
        serializer = ArticleSerializer(data=request.data)
        if serializer.is_valid(raise_exception=True):
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        # return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

생략 가능해짐

실습

Django REST Framework

- 2939 ~ 2940. 도서 관리 API 만들기
- 2941 ~ 2945. 가수 정보 제공 API 만들기
- 3064. DRF 서버 프로젝트와 app 생성
- 3065. DRF 서버 모델 설정
- 3066. DRF serializer 생성하기

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 REST API에서 '자원'을 식별하는 방법으로 가장 알맞은 것은?

a) HTTP Method
b) JSON
c) URI
d) Template

2 API의 역할로 적절한 설명은?

a) 사용자 인터페이스 디자인
b) 데이터베이스 설계 도구
c) HTML을 웹 브라우저에 출력
d) 두 소프트웨어 간 통신을 가능하게 하는 중간 매개

3 REST API에서 데이터를 가져오는 데 사용되는 HTTP Method는?

a) POST
b) GET
c) PUT
d) DELETE

4 다음 중 JSON의 특징으로 올바르지 않은 것은?

a) 텍스트 기반의 데이터 포맷이다
b) Python에서만 사용된다
c) 키-값 구조를 가진다
d) 다양한 시스템 간 데이터 전달에 적합하다

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 HTTP 응답 코드 중 자원이 성공적으로 생성되었음을 의미하는 코드는?

a) 200
b) 201
c) 204
d) 400

6 Django REST framework에서 직렬화를 담당하는 클래스는?

a) `ModelForm`
b) `ModelViewSet`
c) `ModelSerializer`
d) `JSONParser`

7 Serializer의 many=True 옵션은 어떤 경우에 사용하는가?

a) QuerySet 등 다수의 객체를 직렬화할 때
b) HTML로 데이터를 변환할 때
c) 단일 객체를 직렬화할 때
d) JSON 응답을 수신할 때

8 다음 중 RESTful 설계에 부합하지 않는 URL은?

a) `/articles/`
b) `/articles/1/`
c) `/deleteArticle/1`
d) `/users/12/`

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 DELETE 성공 후 본문 없이 응답할 때의 상태 코드는?

10 인터넷에서 자원을 식별하기 위한 문자열을 무엇이라고 하는가?

11 클라이언트가 요청한 데이터를 서버가 찾지 못했을 때 사용하는 HTTP 상태 코드는?

12 DRF에서 함수형 뷰에 HTTP 메서드를 명시할 때 사용하는 데코레이터는?

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

13 다음 코드 중 여러 개의 Article 객체를 직렬화하기 위한 코드로 적절한 것은?

```
articles = Article.objects.all()
serializer = ArticleSerializer(articles, ________)
```

a) context=True
b) data=True
c) partial=True
d) many=True

14 다음 view 함수에서 개선해야 할 점으로 가장 적절한 것은?

```
@api_view(['POST'])
def article_create(request):
    serializer = ArticleSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data)
```

a) status 인자를 지정하지 않아 응답 코드가 명확하지 않음
b) data가 아니라 instance를 전달해야 함
c) POST 요청이므로 GET으로 변경해야 함
d) is_valid() 뒤에 ()가 빠져서 작동하지 않음

확인 문제

정답 및 해설

1) c) URI 2) d) 두 소프트웨어 간 통신을 가능하게 하는 중간 매개 3) b) GET 4) b) Python에서만 사용된다 5) b) 201

6) c) ModelSerialize 7) a) QuerySet 등 다수의 객체를 직렬화할 때 8) c) /deleteArticle/1

1. REST에서 자원은 URI를 통해 식별됩니다.
2. API는 Application Programming Interface의 약자로, 서로 다른 소프트웨어가 정해진 규칙에 따라 요청하고 응답하며 통신할 수 있도록 해주는 매개체입니다.
3. GET은 서버로부터 데이터를 조회(읽기)할 때 사용하는 메서드입니다.반대로 POST, PUT, DELETE는 각각 생성, 수정, 삭제에 사용됩니다.
4. JSON은 JavaScript Object Notation의 약자로, 언어에 독립적인 포맷입니다. Python뿐 아니라 JavaScript, Java, Go 등 다양한 언어에서 사용 가능합니다.
5. 201 Created는 서버가 요청을 받아 새로운 리소스를 성공적으로 생성했을 때 사용되는 상태 코드입니다.
6. ModelSerializer는 Django 모델을 기반으로 자동으로 필드를 구성해, 데이터를 JSON 등으로 직렬화하거나 입력 데이터를 역직렬화할 때 사용됩니다.
7. many=True는 여러 개의 객체(QuerySet 등)를 직렬화할 때 필요합니다. 단일 객체일 경우는 생략하거나 many=False로 처리합니다.
8. 204 No Content는 요청이 성공적으로 수행되었지만, 응답 본문에 보낼 데이터가 없을 때 사용하는 상태 코드입니다. DELETE 요청 후 결과만 알리고 별도 내용은 전달하지 않을 때 적절합니다.

확인 문제

정답 및 해설

9) 204 10) URI 11) 404 12) @api_view

13) d) many=True 14) a) status 인자를 지정하지 않아 응답 코드가 명확하지 않음

8. ModelSerializer는 Django REST framework에서 사용되는 클래스입니다.
9. HTTP 204 No Content는 클라이언트의 요청이 서버에서 성공적으로 처리되었지만, 응답 본문(Response Body)에 돌려줄 데이터가 없음을 나타내는 성공(2xx) 상태 코드입니다.
10. URI(Uniform Resource Identifier)는 인터넷 상에서 특정 자원(리소스)의 위치나 이름을 식별하기 위한 문자열입니다.
11. @api_view는 Django REST framework에서 함수형 뷰에 사용할 수 있는 HTTP 메서드 목록(GET, POST 등)을 지정하는 데코레이터입니다.
13. Article.objects.all()은 여러 개의 Article 인스턴스를 포함하는 QuerySet입니다. 이처럼 다수의 객체를 Serializer에 넘길 때는 many=True를 지정해야,DRF가 이를 리스트 형태로 처리할 수 있게 됩니다. many=True를 생략하면 DRF는 단일 객체로 처리하려고 하며, 타입 오류가 발생할 수 있습니다.
14. 이 코드 자체는 정상 동작합니다. 하지만 Response()에 상태 코드를 명시하지 않으면 기본값인 200 OK가 반환됩니다.POST 요청에서 새로운 데이터를 생성한 경우에는 명시적으로 201 Created를 반환하는 것이 RESTful한 설계입니다. 따라서 return Response(serializer.data, status=status.HTTP_201_CREATED)로 작성하는 것이 권장됩니다. is_valid()에 괄호가 빠진 경우는 문법 오류로 즉시 실패하며, 이 문제에서는 괄호가 있으므로 해당 사항이 아닙니다.

활동 정리

핵심 키워드

개념	설명	예시
API	두 소프트웨어가 데이터를 주고받을 수 있게 해주는 통신 인터페이스	날씨 앱 ↔ 기상청 API
REST API	자원을 URI로 식별하고 HTTP Method로 동작을 수행하는 API 설계 방식	/articles/1/에 GET 요청 → 게시글 조회
URI	인터넷 자원을 식별하는 고유한 문자열 주소	/users/10/, /articles/3/
HTTP Method	자원에 대한 행위를 지정하는 요청 방식	GET, POST, PUT, DELETE
JSON	시스템 간 데이터 교환에 사용되는 경량 구조의 포맷	{ "title": "Django", "content": "..." }
HTTP Status Code	서버가 요청에 대해 어떤 결과를 반환했는지 알려주는 코드	200 OK, 201 Created, 400 Bad Request, 404 등
Serialization	Python 객체를 JSON 등 외부 시스템과 통신 가능한 형태로 변환하는 과정	Article 모델 → JSON 응답

<표3_Django_DRF 1_핵심 키워드 1>

활동 정리

핵심 키워드

개념	설명	예시
ModelSerializer	Django 모델을 기반으로 직렬화를 자동 처리해주는 DRF 클래스	`ArticleSerializer`
@api_view	함수형 뷰에서 허용할 HTTP Method를 지정하는 데코레이터	`@api_view(['GET', 'POST'])`
Response	DRF에서 응답 데이터를 감싸서 반환하는 객체	`Response(serializer.data, status=201)`
partial	전체 필드가 아닌 일부 필드만 업데이트할 수 있도록 허용하는 인자	`partial=True`
204 No Content	DELETE 성공 후 본문 없이 응답할 때 사용하는 상태 코드	`return Response(status=204)`
fixtures	초기 데이터를 JSON 파일로 정의하여 로드하는 방식	`loaddata articles.json`
migrate	모델 변경사항을 데이터베이스에 반영하는 명령어	`python manage.py migrate`
QuerySet	데이터베이스에서 조회된 객체들의 집합	`Article.objects.all()`

<표4_Django_DRF 1_핵심 키워드 2>

활동 정리

오늘 공부한 내용 요약 및 정리

- **REST API 개념 이해**
  - API는 **서로 다른 소프트웨어 간 통신**을 가능하게 하는 인터페이스입니다.
  - REST API는 **자원 중심의 통신 설계 방식**으로, 자원을 URI로 식별하고 HTTP 메서드 (GET, POST, PUT, DELETE)로 행위를 정의합니다.
  - 응답은 HTML이 아닌 **JSON 형식의 데이터**로 반환되는 것이 특징입니다.

- **RESTful 설계 구성 요소**
  - URI: 자원을 식별하는 주소 (/articles/1/)
  - HTTP Method: 자원에 대한 행위 (조회, 생성, 수정, 삭제)
  - HTTP Status Code: 응답 결과에 대한 상태 표현 (예: 200, 201, 204, 400, 404 등)
  - JSON: 통신을 위한 데이터 표현 형식

- **Django에서 REST API 구현 준비**
  - Django는 기본적으로 HTML을 응답하지만, DRF(Django REST Framework)를 사용하면 JSON 기반의 RESTful API 서버를 구축할 수 있습니다.
  - 가상환경 구성, 패키지 설치, migrate, loaddata 등을 통해 프로젝트 기반을 설정합니다.
  - Postman을 이용해 API 요청 테스트를 수행합니다.

활동 정리

오늘 공부한 내용 요약 및 정리

- DRF 주요 구성 요소
  - ModelSerializer: Django 모델 기반의 자동 직렬화 처리 클래스
  - Serializer: 데이터 구조를 JSON으로 직렬화하거나 입력 값을 검증
  - @api_view 데코레이터: 함수형 뷰에서 HTTP 메서드 제어
  - Response 객체: API 응답을 구성하는 DRF 전용 응답 클래스

- CRUD 구현 흐름
  - GET (List/Detail): 전체 또는 단일 자원 조회
  - POST: 새 자원 생성 (201 Created)
  - DELETE: 자원 삭제 후 204 No Content 응답
  - PUT/PATCH: 전체 또는 일부 자원 수정 → partial=True 여부에 따라 부분 수정 처리 가능

- 실습과 예외 처리
  - 실습에서는 Article 모델을 기준으로 CRUD 전 과정을 연습합니다.
  - Serializer의 .is_valid()에서 발생하는 ValidationError는 클라이언트 입력 오류를 처리하기 위해 필요합니다.
  - raise_exception=True 옵션을 통해 예외 응답을 자동 처리할 수 있습니다.

활동 정리

배달 앱에서 음식을 주문할 때, 우리는 가게에 직접 전화하지 않아도 됩니다!

1. 배달 앱이 **REST API** 를 통해 가게에 주문 정보를 전달하고
2. 가게는 그 정보를 받아서 **JSON** 형식으로 “주문 확인”을 응답합니다.
3. 우리는 어떤 메뉴를 주문했는지 **GET** 요청으로 다시 확인할 수도 있어요

- 다양한 서비스들이 서로 약속된 방법으로 대화하는 방식, REST API를 배워보았습니다.
- 복잡한 설명 없이, 주소(URI)를 정하고, 행동(GET, POST, DELETE)을 선택하고, 결과를 받아보았습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다